



**APRIL 2026**

**KARPAGAM COLLEGE OF ENGINEERING**  
**COIMBATORE – 641 032**



**ANNA UNIVERSITY: CHENNAI 600 025**

**BONAFIDE CERTIFICATE**

**Certified that this project report “Bone Strength Analyzer Using Multimodal Sensors and Hybrid CNN–LSTM Model” is the bonafide work of “ABISHEK B (717822L301), ARUL PANDIAN I (71722L305), KAVINPRABHU M (717822L322), SANKAR KRISHNA P (717822L345)” Who carried out the project work under my supervision.**

**SIGNATURE**

DR. R. SARANKUMAR, ME., PH.D.,

**HEAD OF THE DEPARTMENT**

Professor

Electronics and Communication Engineering  
Karpagam College of Engineering  
Coimbatore - 641032

**SIGNATURE**

MS. R. V. SAYAHNA, M.Tech.

**SUPERVISOR**

Assistant professor

Electronics and Communication Engineering  
Karpagam College of Engineering  
Coimbatore - 641032

---

Certified that the candidate was examined during the viva voce examinations held on \_\_\_\_\_

Signature of the Internal Examiner with date

Signature of External Examiner with date



## **DECLARATION**

**We hereby declare that this Project report entitled “Bone Strength Analyzer Using Multimodal Sensors and Hybrid CNN–LSTM Model” Submitted by me for the degree of BE in Electrical and Electronics Engineering at Karpagam College of Engineering, Coimbatore** is the record of original work done by me under the guidance and supervision of **Ms. R. V. Sayahna, M.Tech., Assistant Professor** at the Department of Electrical and Electronics Engineering, Karpagam College of Engineering, Coimbatore – 641032 and has not formed the basis for the award of any degree, or diploma or titles in this institution or any other Institution of higher learning.

Date :

**Name and Signature of the Candidates**

Place : Coimbatore

**ABISHEK B                      717822L301**

**ARUL PANDIAN I              717822L305**

**KAVIN PRABHU M          717822L322**

**SANKAR KRISHNA P        717822L345**

## ACKNOWLEDGEMENT

First and foremost praises and thanks to the almighty for her showers and blessings throughout our project work to complete it successfully.

We extend our gratitude to the Management of Karpagam College of Engineering, Coimbatore for the excellent infrastructure and support facilities to undergo the project work.

We are very grateful to **Dr. P. KARTHIGAIKUMAR**, the Principal and **Dr. R. SARANKUMAR. M.E., Ph.D.** Head of the Department, **Electronics and Communication Engineering** for provided the facilities, support and permission to carried out our project work at our esteemed institution.

We record my sincere gratitude to our Project Coordinator **Dr. C. PRIYA. M.E., Ph.D.** for giving inputs, encouragement for the continuous improvement during the progress and to complete this project work.

We would like to express our sincere gratitude to our Supervisor **Ms. R. V. Sayahna, M.Tech.** for the continuous support for our UG study, for her motivation and adequate guidance which helped us to achieve success in all our accomplishments and to complete this project work.

We also thank all the teaching faculty members and non-teaching Staff members of the department of Electronics and Communication Engineering, Karpagam College of Engineering, Coimbatore for their kindness and support.

We would like to thank **our parents, family members and friends** who sacrificed their time and energy to complete the project work successfully.

**ABISHEK B**                      **717822L301**

**ARUL PANDIAN I**              **717822L305**

**KAVIN PRABHU M**              **717822L322**

**SANKAR KRISHNA P**          **717822L345**

# TABLE OF CONTENTS

| CHAPTER No. | TITLE                                        | PAGE No. |
|-------------|----------------------------------------------|----------|
|             | <b>ABSTRACT</b>                              | i        |
|             | <b>LIST OF FIGURES</b>                       | ii       |
|             | <b>LIST OF ABBREVIATIONS</b>                 | iv       |
| <b>1.</b>   | <b>INTRODUCTION</b>                          | 5        |
| <b>2.</b>   | <b>SYSTEM DESIGN</b>                         | 8        |
|             | 2.1 Existing System                          | 6        |
|             | 2.2 Proposed System                          | 6        |
|             | 2.3 Block Diagram                            | 7        |
|             | 2.4 Methodology                              | 7        |
| <b>3.</b>   | <b>HARDWARE DESCRIPTION</b>                  | 8        |
|             | 3.1 NODEMCU (ESP8266)                        | 8        |
|             | 3.1.1 Pin diagram of ESP8266                 | 9        |
|             | 3.1.2 Features Of Esp8266                    | 9        |
|             | 3.1.3 Advantages/ Improvements<br>in Esp8266 | 10       |
|             | 3.2 NODEMCU Uno Board Description            | 11       |

|       |                                       |    |
|-------|---------------------------------------|----|
| 3.2.1 | Voltage Regulator                     | 12 |
| 3.2.2 | Crystal Oscillator                    | 12 |
| 3.3   | OLED                                  | 13 |
| 3.3.1 | Introduction                          | 13 |
| 3.3.2 | OLED Module                           | 13 |
| 3.3.3 | Pin Description                       | 14 |
| 3.4   | MPU6050 GYRO SENSOR:                  | 15 |
| 3.4.1 | Pin Configuration                     | 15 |
| 3.4.2 | Description                           | 15 |
| 3.5   | CIRCUIT DIAGRAM AND EXPLANATION       | 16 |
| 3.6   | PRESSURE SENSOR INTERFACING           | 16 |
| 3.6.1 | Components Required                   | 16 |
| 3.7   | RECTIFIER CIRCUIT                     | 20 |
| 3.8   | FLEX SENSOR                           | 22 |
| 3.9   | PIEZO CRYSTAL                         | 24 |
| 3.9.1 | Types of Piezoelectric Sensors        | 25 |
| 3.9.2 | Application in Bone Strength Analyzer | 25 |
| 3.9.3 | Advantages of Piezoelectric Sensors:  | 25 |
| 4.    | <b>SOFTWARE DESCRIPTION</b>           | 27 |

|        |                                       |    |
|--------|---------------------------------------|----|
| 4.1    | ARDUINO DEVELOPMENT ENVIRONMENT       | 27 |
| 4.1.1  | Writing Sketches                      | 27 |
| 4.1.2  | Tabs, Multiple Files, and Compilation | 27 |
| 4.1.3  | Uploading                             | 28 |
| 4.1.4  | Libraries                             | 29 |
| 4.2    | PROGRAMMING                           | 29 |
| 4.3    | PYTHON                                | 31 |
| 4.3.1  | Python 3.7                            | 31 |
| 4.4    | THONNY IDE                            | 37 |
| 5.     | <b>WORKING</b>                        | 40 |
| 5.1    | WORKING OF THE BONE STRENGTH          | 40 |
|        | ANALYZER IN DETAIL                    |    |
| 5.1.1. | Hardware Components                   | 40 |
| 5.1.2. | Data Collection                       | 40 |
| 5.1.3. | Data Processing (NodeMCU)`            | 40 |
| 5.1.4. | Machine Learning Model                | 41 |
| 3.1.5. | Real-Time Feedback and Classification | 41 |
| 5.1.6. | User Interface and Display            | 41 |
| 5.1.7. | Portability and Accessibility         | 42 |
| 5.1.8. | Non-Invasive and Efficient            | 42 |

|           |                                                 |    |
|-----------|-------------------------------------------------|----|
|           | 5.2 SUMMARY                                     | 42 |
|           | 5.3 TABLE OF CONTENTS                           | 46 |
|           | 5.4 PREPROCESSING                               | 49 |
|           | 5.5 IMAGE LABELING AND DATASET<br>DISTRIBUTIONS | 50 |
|           | 5.6 TENSORFLOW                                  | 52 |
|           | 5.7 TENSORFLOW IMPLEMENTATION                   | 53 |
|           | 5.8 LIMITATIONS                                 | 55 |
| <b>6.</b> | <b>RESULT</b>                                   | 57 |
|           | <b>DISCUSSION</b>                               | 57 |
| <b>7.</b> | <b>CONCLUSION</b>                               | 59 |
|           | <b>REFERENCES</b>                               | 60 |
|           | <b>ANNEXURE</b>                                 | 62 |

## **ABSTRACT**

The Bone strength degradation is a silent and progressive condition that significantly increases the risk of fractures, osteoporosis, and long-term mobility impairment. Conventional diagnostic techniques such as dual-energy X-ray absorptiometry (DEXA) provide reliable bone mineral density measurements; however, they are expensive, radiation-based, clinic-dependent, and unsuitable for continuous monitoring. This paper presents a portable, non-invasive bone strength analysis system that integrates multimodal sensor data with a hybrid deep learning model for qualitative bone health assessment. The proposed system combines inertial motion sensing, pressure response measurement, and optional bone imaging to capture complementary mechanical and structural characteristics of bone tissue. Motion and vibration signals are acquired using an MPU6050 inertial sensor, while force response data are obtained through a piezoelectric transducer. Bone surface or radiographic images provide additional spatial information. A hybrid Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) model is employed to extract spatial and temporal features, respectively. Feature-level fusion enables robust classification of bone strength into clinically meaningful categories. The system emphasizes portability, affordability, and radiation-free operation, making it suitable for continuous bone health monitoring in both clinical and personal healthcare environments.

## LIST OF FIGURES

| FIG NO | FIGURE NAME                           | PAGE NO |
|--------|---------------------------------------|---------|
| 2.3.1  | Block Diagram                         | 7       |
| 3.2.1  | ESP8266                               | 8       |
| 3.1.2  | Pin Diagram of ESP8266                | 9       |
| 3.2.1  | NODEMCU                               | 12      |
| 3.3.2  | OLED Module                           | 13      |
| 3.3.1  | Pin Diagram                           | 14      |
| 3.4.1  | MPU6050 Gyro Sensor                   | 15      |
| 3.4.2  | Setup(1)                              | 16      |
| 3.4.3  | Setup(2)                              | 16      |
| 3.5.1  | Circuit Diagram                       | 17      |
| 3.6.1  | Circuit Diagram & Working Explanation | 17      |
| 3.6.2  | Voltage                               | 20      |
| 3.8.1  | Flex sensor                           | 22      |
| A.1    | Conference Acceptance Mail            | 62      |

## **LIST OF ABBREVIATIONS**

|      |                              |
|------|------------------------------|
| OLED | ORGANIC LIGHT EMITTING DIODE |
| FLEX | FLEXIBLE POTENTIOMETER       |
| PCB  | PRINTED CIRCUIT BOARD        |

# CHAPTER 1

## INTRODUCTION

Bone health is essential for maintaining mobility, posture, and overall physical well-being. With the growing prevalence of bone-related conditions such as osteoporosis, fractures, and age-related bone deterioration, the need for accessible and frequent bone strength assessment tools is greater than ever. Conventional diagnostic methods like bone density (BMD) tests and DEXA scans, though accurate, remain expensive, hospital-dependent, and unsuitable for routine or remote monitoring due to their invasive nature and reliance on specialized equipment.

The Bone Strength Analyzer project is developed to bridge these limitations by offering a portable, non-invasive, and cost-effective device capable of evaluating bone strength in real time. The system incorporates modern sensor technologies—primarily the MPU6050 sensor (for measuring motion, angular displacement, and vibration) and a piezoelectric crystal (for detecting pressure or force responses). These sensors interface with a Node MCU microcontroller, which collects, processes, and transmits data wirelessly.

At the heart of the system lies the K-Nearest Neighbors (KNN) algorithm, a machine learning classification method known for simplicity and reliability. Using real-time sensor data, KNN categorizes bone strength into predefined levels—such as *low*, *medium*, or *high*—allowing users to instantly understand their bone condition without clinical visits. This approach transforms bone assessment from a hospital-only process into an everyday health check.

The device is designed to be user-friendly, making it suitable for elderly individuals, athletes, patients recovering from fractures, and people in remote or underserved regions. Its non-invasive operation, lack of radiation exposure, and ease of repeated use enable continuous monitoring of bone health, early detection of risks, and timely medical intervention. By integrating sensor technology and machine learning, the Bone Strength Analyzer promotes preventive healthcare, supports personalized health

decisions, and reduces dependence on costly clinical infrastructure.

The affordability and portability of the device also make it a strong candidate for mHealth (mobile health) applications, enabling widespread screening programs and community-level health initiatives. The project exemplifies how embedded systems, combined with machine learning, can create practical solutions that improve public health outcomes.

Embedded systems are specialized computing units designed to perform dedicated functions within larger devices. Unlike general-purpose computers, embedded systems focus on specific tasks and emphasize reliability, real-time performance, compactness, and energy efficiency. They are integral to countless modern technologies—from home appliances, smart electronics, and automobiles to medical instruments and industrial machinery.

An embedded system consists of a combination of hardware (microcontrollers, sensors, actuators, communication modules) and software (firmware, RTOS, drivers) that work together to achieve the system's intended functionality. Microcontrollers serve as the computational core, executing instructions, managing peripherals, and ensuring precise timing. These components are selected and optimized based on application-specific needs such as processing capability, power consumption, and available memory. Software development for embedded systems often involves writing firmware that directly controls hardware behavior. Many systems require real-time operating systems (RTOS) to guarantee timely execution, especially in safety-critical applications like automotive control units, medical devices, and aerospace systems. The hardware-software coordination within embedded systems is what enables devices to respond to inputs, perform tasks reliably, and operate autonomously.

The rise of the Internet of Things (IoT) has significantly expanded the importance of embedded systems. They now form the foundation for connected devices that communicate, collect data, and operate intelligently across networks. With advancements in microcontroller technology, embedded systems are increasingly integrating artificial intelligence and machine learning capabilities, enabling smart automation and predictive decision-making.

## **CHAPTER 2**

### **SYSTEM DESIGN**

#### **2.1 Existing System**

Current bone strength assessment methods predominantly rely on clinical techniques such as dual-energy X-ray absorptiometry (DEXA) and bone mineral density (BMD) tests. While these methods are accurate and widely used, they are often limited by their high cost, invasiveness, and reliance on specialized equipment available only in clinical settings. DEXA scans, for instance, expose patients to small doses of radiation and can only be performed periodically, making them unsuitable for continuous monitoring.

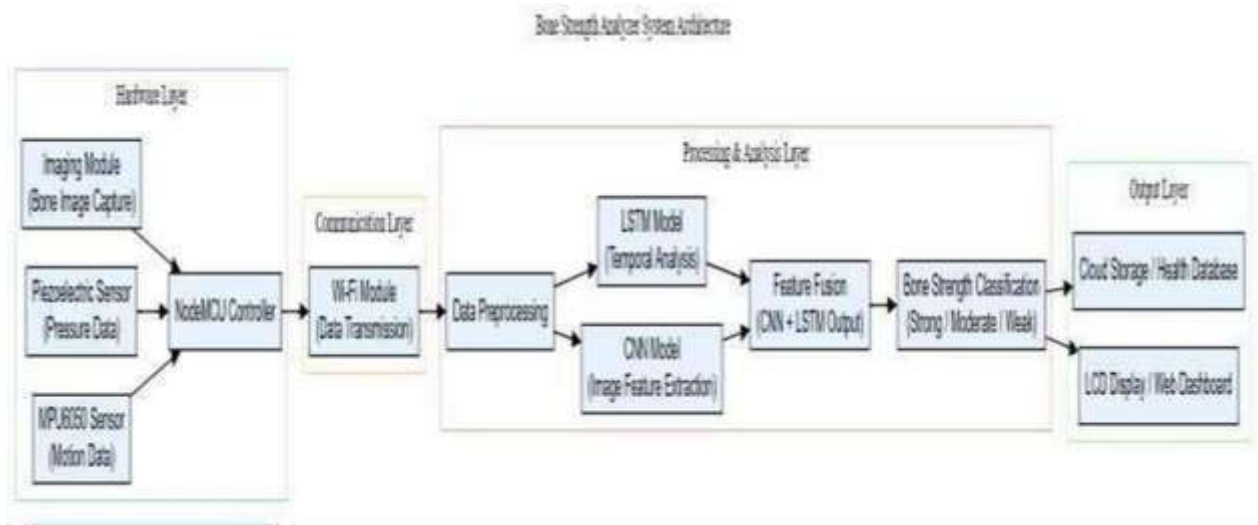
Additionally, these tests require healthcare professionals to operate the equipment and interpret the results, restricting accessibility for individuals in remote or resource-constrained areas. As a result, regular bone health monitoring is not feasible for most individuals, and early detection of bone deterioration often occurs too late to prevent injury.

#### **2.2 Proposed System:**

The proposed system aims to develop an intelligent, portable bone strength analyzer that integrates both hardware sensing and deep learning analysis for accurate bone health assessment. The system consists of a NodeMCU controller interfaced with an MPU6050 sensor and a piezoelectric crystal to capture motion, vibration, and pressure response data from the bone region. Additionally, an imaging module is used to capture bone surface or X-ray images. The collected data are transmitted wirelessly to a computing platform, where a hybrid CNN–LSTM deep learning model processes the multimodal inputs. The CNN extracts spatial and textural features from the bone images, while the LSTM analyzes the temporal characteristics of the sensor signals. Based on the fused analysis, the model

classifies bone strength into categories such as *Strong*, *Moderate*, or *Weak*. The results are displayed in real time on an LCD interface or web dashboard, allowing users and medical professionals to monitor bone integrity conveniently. This system provides a non-invasive, low-cost, and efficient method for early detection of bone weakness and preventive healthcare monitoring.

## 2.2 BLOCK DIAGRAM



**Fig 2.3.1 Block Diagram**

## 2.3 Methodology:

The methodology of the proposed Bone Strength Analyzer system involves the integration of hardware-based sensing and deep learning algorithms to assess bone strength accurately. Initially, the MPU6050 sensor captures acceleration and angular motion data, while the piezoelectric sensor records pressure and vibration responses from the bone region under test. Simultaneously, an imaging module captures bone surface or X-ray images to provide visual information about bone structure. All sensor and image data are collected by the Node MCU controller and transmitted via Wi-Fi to a cloud or local processing system. The data undergo preprocessing, including noise filtering, normalization, and feature extraction.

## CHAPTER 3

### HARDWARE DESCRIPTION

#### 3.1 NODEMCU (ESP8266)



**Fig 3.2.1 ESP8266**

The Atmel AVR® core combines a rich instruction set with 32 general purpose working registers. All the 32 registers are directly connected to the Arithmetic Logic Unit (ALU), allowing two independent registers to be accessed in a single instruction executed in one clock cycle. The resulting architecture is more code efficient while achieving throughputs up to ten times faster than conventional CISC microcontrollers. The ESP8266/P provides the following features: 32Kbytes of In-System Programmable Flash with Read-While- Write capabilities, 1Kbytes EEPROM, 2Kbytes SRAM, 23 general purpose I/O lines, 32 general purpose working registers, Real Time Counter (RTC), three flexible Timer/Counters with compare modes and PWM, 1 serial programmable USARTs , 1 byte-oriented 2-wire Serial Interface (I2C), a 6- channel 10-bit ADC (8 channels in TQFP and QFN/MLF packages) , a programmable Watchdog Timer with internal Oscillator, an SPI serial port, and six software selectable power saving modes.

This allows very fast start-up combined with low power consumption. In Extended Standby mode, both the main oscillator and the asynchronous timer continue to run. Atmel offers the Q Touch® library for embedding capacitive touch buttons,

sliders and wheels functionality into AVR microcontrollers. The patented charge-transfer signal acquisition offers robust sensing and includes fully de-bounced reporting of touch keys and includes Adjacent Key Suppression® (AKS™) technology for unambiguous detection of key events. The easy-to-use Q Touch Suite tool chain allows you to explore, develop and debug your own touch applications. The device is manufactured using Atmel's high density non-volatile memory technology. The On-chip ISP Flash allows the program memory to be reprogrammed In-System through an SPI serial interface, by a conventional non-volatile memory programmer, or by an On-chip Boot program running on the AVR core.

### 3.1.1 Pin diagram of ESP8266:

The ESP8266/P is supported with a full suite of program and system development tools including: C Compilers, Macro Assemblers, and Program Debugger/Simulators, In- Circuit Emulators, and Evaluation kits. Below figure 3.2.1 pin diagram of ESP8266.

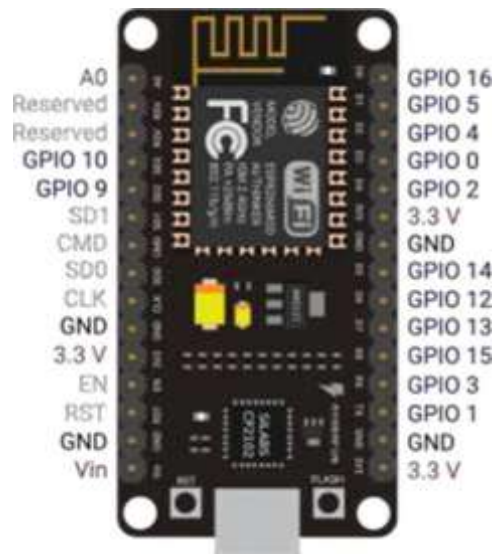


Fig 3.1.2 Pin Diagram Of Esp8266

### 3.1.2 FEATURES OF ESP8266:

- 28-pin Microcontroller
- Flash Program Memory: 32 kilo bytes
- EEPROM Data Memory: 1 kilo bytes
- SRAM Data Memory: 2 kilo bytes
- I/O Pins: 23

- Timers: Two 8-bit / One 16-bit
- A/D Converter: 10-bit Six Channel
- PWM: Six Channels
- RTC: Yes with Separate Oscillator
- MSSP: SPI and I<sup>2</sup>C Master and Slave Support
- USART: Yes
- External Oscillator: up to 20MHz

### 3.1.3 ADVANTAGES/ IMPROVEMENTS IN ESP8266

**Wide voltage range (1.8–5 V)** makes it compatible with both legacy 5 V systems and modern low-voltage designs—rare among microcontrollers.

**Efficient architecture and strong GCC support**, offering high instruction throughput and avoiding proprietary compilers.

**Low-power “PA” variants and solid peripheral set**, including Q Touch capability and micro-amp sleep modes.

#### **Port B (PB [7:0]) XTAL1/XTAL2/TOSC1/TOSC2**

Port B is an 8-bit bi-directional I/O port with internal pull-up resistors (selected for each bit). The Port B output buffers have symmetrical drive characteristics with both high sink and source capability. As inputs, Port B pins that are externally pulled low will source current if the pull-up resistors are activated. The Port B pins are tri-stated when a reset condition becomes active, even if the clock is not running.

Depending on the clock selection fuse settings, PB7 can be used as output from the inverting Oscillator amplifier. If the Internal Calibrated RC Oscillator is used as chip clock source, PB [7:6] is used as TOSC [2:1] input for the Asynchronous Timer/Counter2 if the AS2 bit in ASSR is set. Port C (PC [5:0]) Port C is a 7-bit bi-directional I/O port with internal pull-up resistors (selected for each bit). The PC [5:0] output buffers have symmetrical drive characteristics with both high sink and source capability. As inputs, Port C pins that are externally pulled low will source

current if the pull-up resistors are activated. The Port C pins are tri-stated when a reset condition becomes active, even if the clock is not running.

#### **PC6/RESET**

If the RSTDISBL Fuse is programmed, PC6 is used as an I/O pin. Note that the electrical characteristics of PC6 differ from those of the other pins of Port C.

If the RSTDISBL Fuse is un-programmed, PC6 is used as a Reset input. A low level on this pin for longer than the minimum pulse length will generate a Reset, even if the clock is not running. Shorter pulses are not guaranteed to generate a Reset.

The various special features of Port ‘C’ are elaborated in the Alternate Functions of Port C section.

#### **Port D (PD [7:0])**

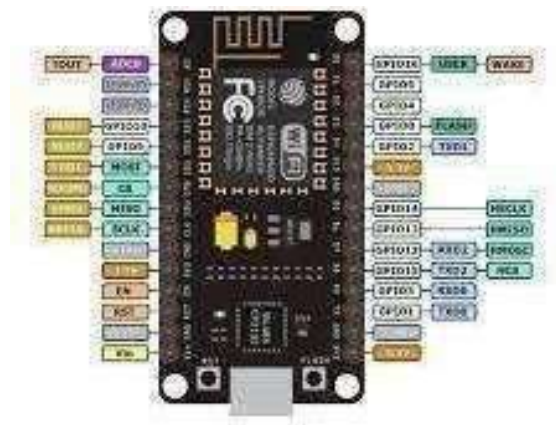
Port D is an 8-bit bi-directional I/O port with internal pull-up resistors (selected for each bit). The Port D output buffers have symmetrical drive characteristics with both high sink and source capability. As inputs Port D pins that are externally pulled low will source current if the pull-up resistors are activated. The Port D pins are tri-stated when a reset condition becomes active, even if the clock is not running.

#### **AVCC**

AVCC is the supply voltage pin for the A/D Converter, PC [3:0], and PE [3:2]. It should be externally connected to VCC, even if the ADC is not used. If the ADC is used, it should be connected to VCC through a low-pass filter. Note that PC [6:4] use digital supply voltage, VCC.

### **3.2 NODEMCU Uno Board Description:**

We will learn about the different components on the NODEMCU board. We will study the NODEMCU UNO board because it is the most popular board in the NODEMCU board family. In addition, it is the best board to get started with electronics and coding. Some boards look a bit different from the one given below figure 3.2.2, but most NODEMCUs have majority of these components in common.



**Fig 3.2.1 Node MCU development board**

NODEMCU development board can be powered by using the USB cable from computer. All we need to do is connect the USB cable to the USB connection (1).

### **Power (Barrel Jack)**

NODEMCU development boards can be powered directly from the AC mains power supply by connecting it to the Barrel Jack.

### **3.2.1 Voltage Regulator**

The function of the voltage regulator is to control the voltage given to the NODEMCU development board and stabilize the DC voltages used by the processor and other elements.

### **3.2.2 Crystal Oscillator**

The crystal oscillator helps NODEMCU development board in dealing with time issues. How does NODEMCU development board, calculate time? The answer is, by using the crystal oscillator. The number printed on top of the NODEMCU crystal is 16.000H9H. It tells us that the frequency is 16,000,000 Hertz or 16 MHz

NODEMCU development board Reset

We can reset were NODEMCU development board, i.e., start were program from the beginning. We can reset the UNO board in two ways. First, by using the reset button (17) on the board. Second, we can connect an external reset button to the NODEMCU development board pin labelled RESET (5).

### **Pins (3.3, 5, GND, Vin)**

- 3.3V (6) – Supply 3.3 output volt

- 5V (7) – Supply 5 output volt

Most of the components used with NODEMCU development board works fine with 3.3 volt and 5 volt. GND (8) (Ground) – There are several GND pins on the NODEMCU development board, any of which can be used to ground were circuit.

Vin (9) – This pin also can be used to power the NODEMCU development board from an external power source, like AC mains power supply.

### **Analog pins:**

The NODEMCU development board has five analog input pins A0 through A5. These pins can read the signal from an analog sensor like the humidity sensor or temperature sensor and convert it into a digital value that can be read by the microprocessor.

## **3.3 OLED**

### **3.3.1 Introduction**



**Fig 3.3.1 OLED Module**

### **3.3.2 OLED Module**

The OLED module shown in the above image is a very popular module available in the market. There are many variants of this module available in the market, having different resolutions, communication protocol or pixel colors. These OLED modules are driven by SSD1306 IC which is a driver IC for 128x64 Dot Matrix OLED segments. The SSD1306 has its own controller and supports both SPI and I2C communication

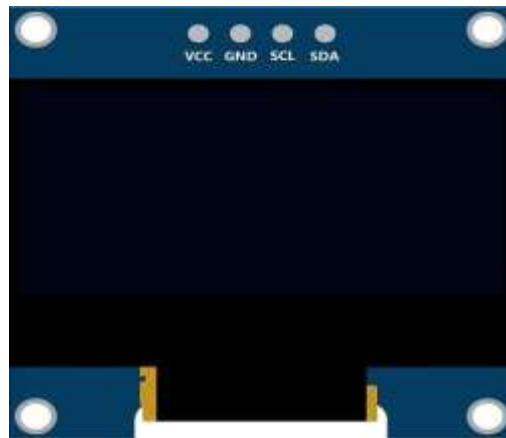
protocols.

Hence, there are various OLED modules in the market, some that support only SPI communication, some that support only I2C communication, and some that support both I2C and SPI communication. (Different number of pins for different modules)

Since the driver IC supports 128x64 resolution, there are some variants that have lesser resolution like 128x32. Different modules support different colors like blue, yellow, white. Some modules support multiple colors as well. You will need to check the specifications of your display module to know which colors are supported. We are using 4-pin I2C supported 128x64 OLED module similar to the one shown in the above image.

For more information about OLED and how to use it, refer the topic [SSD1306 OLED Display](#) in the sensors and modules section.

### 3.3.3 Pin Description



**Fig 3.3.1 Pin Diagram**

The above image shows a 128x64 I2C based OLED module.

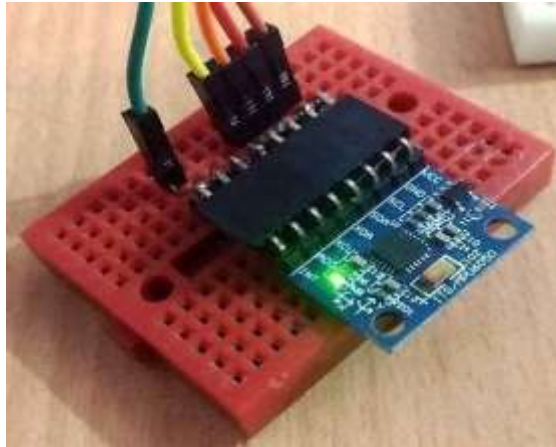
**VCC:** This is the power pin for the module. A supply of 3.3V or 5V can be provided to this pin to power the display.

**GND:** This is the ground pin for the module.

**SCL and SDA:** These are the serial clock and serial data pins for I2C communication.

### 3.4 MPU6050 Gyro Sensor:

MPU-6050 is an 8 pin 6 axis gyro and accelerometer in a single chip. This module works on I2C serial communication by default but it can be configured for SPI interface by configuring its register. For I2C this has SDA and SCL lines. Almost all the pins are multifunctioning but here we are proceeding only with I2C mode pins.



**Fig 3.4.1 MPU6050 Gyro Sensor**

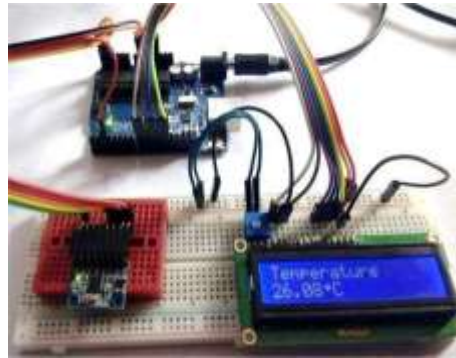
#### 3.4.1 Pin Configuration:

- Vcc:- this pin is used for powering the MPU6050 module with respect to ground GND:- this is ground pin
- SDA:-SDA pin is used for data between controller and mpu6050 module SCL:- SCL pin is used for clock input
- XDA:- This is sensor I2C SDA Data line for configuring and reading from external sensors ((optional) not used in our case)
- XCL:- This is sensor I2C SCL clock line for configuring and reading from external sensors ((optional) not used in our case)
- ADO:- I2C Slave Address LSB (not applicable in our case) INT:- Interrupt pin for indication of data ready.

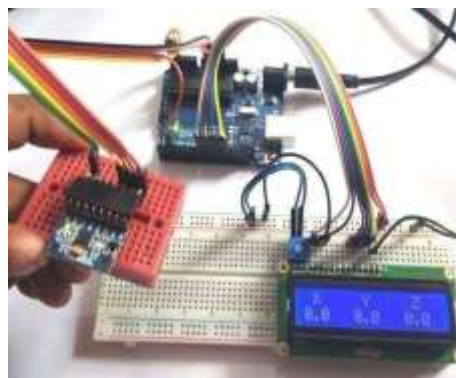
#### 3.4.2 Description:

In this article, we are showing temperature, gyro and accelerometer readings over LCD using MPU6050 with Arduino. This module gives us raw values and normalized

values in output but row values are not stable so here we have showing normalized values over LCD. If you just want accelerometer value, you can also use [Accelerometer ADXL335](#) with Arduino. In this project, we have first shown a temperature value over LCD and after 10 seconds we show gyro values and after 10 seconds we have accelerometer readings as shown in the images below:



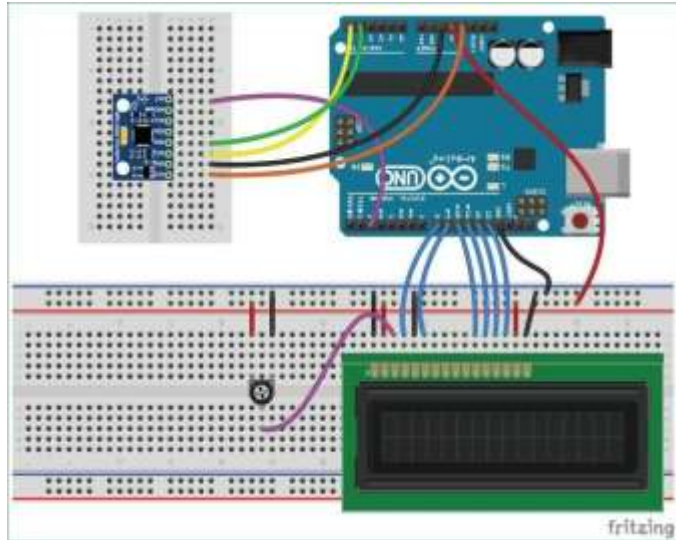
**Fig 3.4.2 setup(1)**



**Fig 3.4.3 setup(2)**

### **3.5 Circuit Diagram and Explanation:**

The circuit diagram, for interfacing MPU6050 with Arduino, is very simple here we have used an LCD and MPU6050. And here we have used a [laptop USB power supply](#). A 10k pot is used for controlling the brightness of the LCD. In connection with MPU6050, we have done 5 connections in which we have connected the 3.3v power supply and ground of MPU6050 to the 3.3v and ground of Arduino. SCL and SDA pins of MPU6050 is connected with Arduino's A4 and A5 pin. And INT pin of MPU6050 is connected to interrupt 0 of Arduino (D2). LCD's RS, RW and EN are directly connected to 8, ground and 9 of Arduino. Data pin are directly connected to digital pin number 10, 11, 12 and 13.



**Fig 3.5.1 Circuit Diagram**

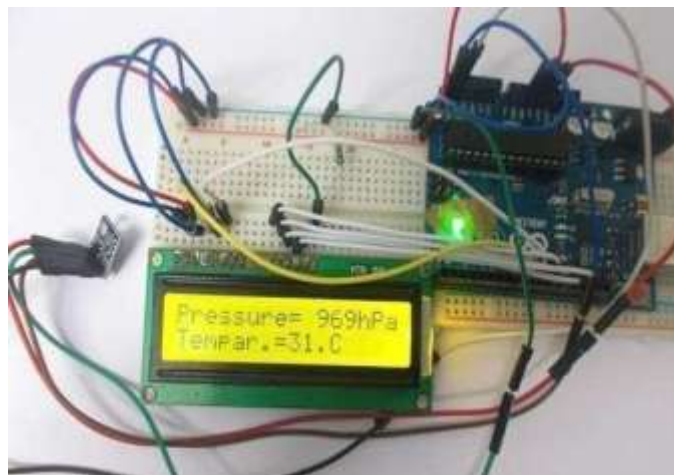
### **3.6 PRESSURE SENSOR INTERFACING:**

#### **3.6.1 Components Required :**

Hardware: Arduino uno board, connecting pins, 220Ω resistor, BMP180 Barometric Pressure Sensor, 16x2 LCD, bread board.

Software: Arduino nightly

#### **Circuit Diagram & Working Explanation**



**Fig 3.6.1 Circuit Diagram & Working Explanation**

After calling for header we don't need to worry for establishing communication between Arduino Uno and BMP180 sensor. We can simply call in special functions which will do that for us. We only need to Initialize an LCD and show the called values from SENSOR on it.

In 16x2 LCD there are 16 pins over all if there is a back light, if there is no back light there will be total 14 pins. One can power or leave the back light pins. Now in the 14 pins there are 8 data pins (7-14 or D0-D7), 2 power supply pins (1&2 or VSS&VDD or GND&+5v), 3rd pin for contrast control (VEE-controls how thick the characters should be shown) and 3 control pins (RS&RW&E).

In the circuit, you can observe that I have only took two control pins, the contrast bit and READ/WRITE are not often used so they can be shorted to ground. This puts LCD in highest contrast and read mode. We just need to control ENABLE and RS pins to send characters and data accordingly.

The connections which are done for LCD are given below:

- PIN1 or VSS to ground
- PIN2 or VDD or VCC to +5v power
- PIN3 or VEE to ground (gives maximum contrast best for a beginner)
- PIN4 or RS (Register Selection) to PIN8 of ARDUINO UNO
- PIN5 or RW (Read/Write) to ground (puts LCD in read mode eases the communication for user)
- PIN6 or E (Enable) to PIN9 of ARDUINO UNO
- PIN11 or D4 to PIN10 of ARDUINO UNO
- PIN12 or D5 to PIN11 of ARDUINO UNO
- PIN13 or D6 to PIN12 of ARDUINO UNO
- PIN14 or D7 to PIN13 of ARDUINO UNO

The ARDUINO IDE allows the user to use LCD in 4 bit mode. This type of communication enables the user to decrease the pin usage on ARDUINO, unlike other the ARDUINO need not be programmed separately for using it in 4 bit mode because by default the ARDUINO is set up to communicate in 4 bit mode. In the circuit you can see we have used 4bit communication (D4-D7). [Also check: Interface LCD with Arduino Uno]. So from mere observation from above table we are connecting 6 pins of LCD to

controller in which 4 pins are data pins and 2 pins for control.

### **POWER SUPPLY:**

This document presents the solution for a 12V 1A fly back converter based on the Infineon OPTIREG™ TLE8386-2EL controller and IPD50N08S4-13 OptiMOS™-T2. The user is guided through the component selections, the circuit design and, finally, an overview of the experimental results are presented. The TLE8386-2EL is part of the Automotive OPTIREG™ family and it implements a low-side-sense current mode controller with built in protection features. The device is AECQ-100 qualified. The IPD50N08S4-13 is an AEC-Q101 qualified 80V N-channel enhanced mode MOSFET, it is part of the OptiMOS™-T2 family. Intended audience This document is intended for power supply design engineers, application engineers, students, etc., who need to design a Fly back converter for automotive power applications where a galvanic isolation between two voltage domains is required. In particular the focus is on a battery connected fly back that delivers up to 12W at 12V output voltage; the intention is to provide the user with all of the needed information to fully design and characterize the SMPS bringing it from an engineering concept to its production.

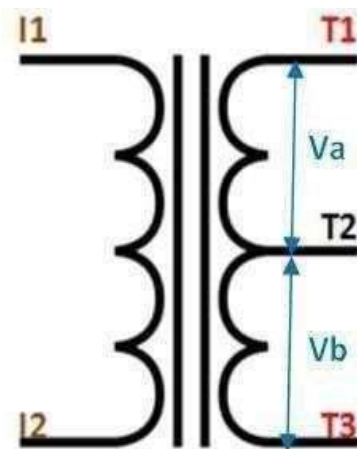
Specific features and applications are: - 48V to 12V Automotive applications Isolated current mode SMPS - Fly back regulators with auxiliary sensing

- Centre Tapped Transformer Specifications
- Step-down Centre tapped Transformer
- Input Voltage: 220V AC at 50Hz
- Output Voltage: 24V, 12V or 0V
- Output Current: 1A
- Vertical mount type
- Low cost and small package

A centre-tapped transformer also known as two phase three wire transformer is normally used for rectifier circuits. When a digital project has to work with AC mains a Transformer is used to step-down the voltage (in our case, to 24V or 12V) and then convert it to DC by using a rectifier circuit. In a center-tapped transformer the peak

inverse voltage is twice as in bridge rectifier hence this transformer is commonly used in full wave rectifier circuits. The operation and theory behind a Center tapped transformer is very similar to a normal secondary transformer. A primary voltage will be induced in the primary coil (I1 and I3) and due to magnetic induction the voltage will be transferred to the secondary coil. Here in the secondary coil of a center tapped transformer, there will be an additional wire (T2) which will be placed exactly at the center of the secondary coil, hence the voltage here will always be zero. If we combine this zero potential wire (T2) with either T1 or T2, we will get a voltage of 12V AC. If this wire is ignored and voltage across T1 and T2 is considered then we will get a voltage of 24V AC. This feature is very useful for the function of a full wave rectifier.

Let us consider the voltage given by the first half of the secondary coil as  $V_a$  and the voltage across the second half of the secondary coil as  $V_b$  as shown



**Fig 3.6.2 Voltage**

### **3.7 RECTIFIER CIRCUIT:**

We have learnt in rectifier circuits about converting a sinusoidal ac voltage into its corresponding pulsating dc. Apart from the dc component, this pulsating dc voltage will have unwanted ac components like the components of its supply frequency along with its harmonics (together called ripples). These ripples will be the highest for a single-phase half wave rectifier and will reduce further for a single-phase full wave rectifier. The ripples will be minimum for 3-phase rectifier circuits. Such supply is not useful for driving complex electronic circuits. For most supply purposes constant dc voltage is required than the pulsating output of the rectifier. For most applications the

supply from a rectifier will make the operation of the circuit poor. If the rectifier output is smoothened and steady and then passed on as the supply voltage, then the overall operation of the circuit becomes better. Thus, the output of the rectifier has to be passed through a filter circuit to filter the ac components. The filter is a device that allows passing the dc component of the load and blocks the ac component of the rectifier output. Thus the output of the filter circuit will be a steady dc voltage. The filter circuit can be constructed by the combination of components like capacitors, resistors, and inductors. Inductor is used for its property that it allows only dc components to pass and blocks ac signals.

Capacitor is used so as to block the dc and allows ac to pass. All the combinations and their working are explained in detail below. Series Inductor Filter The circuit diagram of a full wave rectifier with a series inductor filter is given below. As the name of the filter circuit suggests, the Inductor L is connected in series between the rectifier circuit and the load. The inductor carries the property of opposing the change in current that flows through it. In other words, the inductor offers high impedance to the ripples and no impedance to the desired dc components. Thus the ripple components will be eliminated. When the rectifier output current increases above a certain value, energy is stored in it in the form of a magnetic field and this energy is given up when the output current falls below the average value.

Thus all the sudden changes in current that occurs in the circuit will be smoothened by placing the inductor in series between the rectifier and the load. The waveform below shows the use of inductor in the circuit. From the circuit, for zero frequency dc voltage, the choke resistance  $R_i$  in series with the load resistance  $R_L$  forms a voltage divider circuit, and thus the dc voltage across the load is  $V_{dc} = \frac{R_L}{(R_i + R_L)} V_{dc}$  is the output from a full wave rectifier. In this case, the value of  $R_i$  is negligibly small when compared to  $R_L$ . The effect of higher harmonic voltages can be easily neglected as better filtering for the higher harmonic components take place. This is because of the fact that with the increase in frequency, the reactance of the inductor also

increases. It should be noted that a decrease in the value of load resistance or an increase in the value of load current will decrease the amount of ripples in the circuit. So, the series inductor filter is mostly used in cases of high load current or small load resistance. A simple series inductor filter may not be properly used. It is always better to use a shunt capacitor (C) with series inductor (L) to form an LC Filter. Shunt Capacitor Filter As the name suggests, a capacitor is used as the filter and this high value capacitor is shunted or placed across the load impedance.

This capacitor, when placed across a rectifier gets charged and stores the charged energy during the conduction period. When the rectifier is not conducting, this energy charged by the capacitor is delivered back to the load.

Through this energy storage and delivery process, the time duration during which the current flows through the load resistor gets increased and the ripples are decreased by a great amount. Thus for the ripple component with a frequency of 'f' megahertz, the capacitor 'C' will offer a very low impedance. The value of this impedance can be written as: Shunt Capacitor Impedance =  $1/2\pi fC$  Thus the dc components of the input signal along with the few residual ripple components, is only allowed to go through the load resistance R Load. The high amount of ripple components of current gets bypassed through the capacitor C. Now let us look at the working of Half-wave rectifier and Full- wave rectifier with Capacitor filters, their output filtered waveform, ripple factor, merits and demerits in detail.

### 3.8 FLEX SENSOR



**Fig 3.8.1 FLEX SENSOR**

A flex sensor is a type of sensor that measures the amount of bending or flexing in a given material. It is typically a thin, flexible strip embedded with conductive materials that change resistance when the strip is bent. Flex sensors are commonly used in applications that require monitoring of physical movement, deformation, or angle, making them ideal for wearable devices, robotics, and even in medical diagnostics for tracking joint movement. When the sensor bends, its resistance increases or decreases in proportion to the degree of flex, which can be measured by a microcontroller or any signal processing system.

### **Working Principle of a Flex Sensor:**

Flex sensors operate based on the principle of variable resistance. As the sensor bends or flexes, its internal conductive particles are displaced, causing changes in the electrical resistance of the sensor. In its unbent state, the flex sensor has a baseline resistance.

When bent, the sensor's conductive ink particles are further separated, which increases the resistance. This change in resistance is proportional to the amount of bending, allowing the sensor to measure how much an object has flexed. The output signal is typically an analog voltage, which can be easily read and interpreted by microcontrollers like Arduino or Node MCU.

### **Application in Bone Strength Analyzer:**

In the context of a Bone Strength Analyzer, a flex sensor could be utilized to measure the deformation or flexion of bones or joints in real-time. For example, it can be attached to a body part (such as a limb) to track the angle of joint movement, which can provide insights into bone and joint flexibility. When combined with other sensors like the MPU6050 (which measures motion and angular velocity), the flex sensor can enhance the device's ability to analyze the structural integrity and strength of bones by recording how much a bone or joint can bend under specific conditions. This data can be particularly useful in assessing conditions like osteoporosis, where bone flexibility and resistance to bending forces are critical indicators of bone health.

The flex sensor offers a non-invasive and continuous monitoring option, allowing users to assess bone or joint health regularly. It can be used to track rehabilitation progress after a bone injury or surgery, helping medical professionals measure improvements in bone flexibility over time. Moreover, the flex sensor's low power consumption and lightweight design make it ideal for portable devices, ensuring that it can be easily integrated into wearable or handheld medical devices without compromising comfort or functionality.

In summary, the flex sensor plays a critical role in monitoring physical deformation, offering a simple yet effective way to measure how bones and joints react to pressure and movement. This data can be vital in understanding the overall health of the skeletal system and can be integrated into advanced bone strength analysis systems.

### **3.9 PIEZO CRYSTAL :**

A piezoelectric sensor is a device that utilizes the piezoelectric effect to convert mechanical energy, such as pressure or vibration, into electrical energy. The piezoelectric effect is a phenomenon observed in certain materials (like quartz, ceramics, and certain polymers) that generate an electrical charge when subjected to mechanical stress. This property makes piezoelectric sensors highly effective for a wide range of applications, including force measurement, pressure sensing, and dynamic acceleration detection.

#### **Working Principle of Piezoelectric Sensors:**

The core principle behind piezoelectric sensors involves the generation of electrical charges in response to mechanical deformation. When pressure, vibration, or stress is applied to a piezoelectric material, its internal dipoles become polarized, resulting in an electrical charge across the material's surface. This charge can be measured and converted into a voltage signal using appropriate circuitry. The output of a piezoelectric sensor is typically proportional to the amount of mechanical stress applied, making it highly sensitive to dynamic changes.

### **3.9.1 Types of Piezoelectric Sensors:**

1. **Force Sensors:** These sensors are designed to measure the force or load applied to an object. They are commonly used in applications such as load cells for weighing systems and industrial automation.
2. **Pressure Sensors:** Piezoelectric pressure sensors are widely used in medical devices, automotive applications, and environmental monitoring, as they can accurately measure pressure changes in fluids and gases.
3. **Vibration Sensors:** These sensors are used to detect vibrations in machinery and structures, making them essential for condition monitoring in industrial equipment and structural health assessments

### **3.9.2 Application in Bone Strength Analyzer:**

In the context of the Bone Strength Analyzer, piezoelectric sensors play a crucial role in measuring the pressure applied to bones during activities or assessments. When a force is exerted on a bone, the piezoelectric sensor detects this pressure and converts it into an electrical signal. This data is vital for understanding how bones respond to stress, which is an important indicator of bone strength and integrity.

The sensor can be positioned directly on a body part or embedded within a wearable device to continuously monitor the pressure experienced by the bones during physical activity. By capturing real-time pressure data, the Bone Strength Analyzer can provide insights into the bone's ability to withstand external forces, helping to assess conditions like osteoporosis, where bone density and strength are compromised.

Moreover, the piezoelectric sensor is beneficial for its high sensitivity and quick response time, allowing for immediate feedback on bone health. This capability enables users to track changes in bone strength over time and to identify potential issues before they lead to injury or fractures. The non-invasive nature of piezoelectric sensors ensures that users can wear the device comfortably during their daily activities without disruption.

### **3.9.3 Advantages of Piezoelectric Sensors:**

- **High Sensitivity:** Piezoelectric sensors can detect even minute changes in pressure or

force, making them suitable for precise measurements.

- **Wide Frequency Response:** These sensors can operate effectively across a broad range of frequencies, allowing them to capture dynamic changes quickly.
- **Compact Size:** Piezoelectric sensors are generally small and lightweight, making them easy to integrate into portable and wearable devices.
- **Non-Invasive Measurement:** They offer a non-invasive way to monitor pressure and force, which is particularly advantageous in medical and health-related applications.

In summary, piezoelectric sensors are essential components in devices that require the monitoring of mechanical stress and pressure, such as the Bone Strength Analyzer. Their ability to convert mechanical energy into electrical signals allows for accurate and real-time assessment of bone strength, contributing to effective bone health management.

## CHAPTER 4

### SOFTWARE DESCRIPTION

#### 4.1 Arduino Development Environment

The Arduino development environment contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions, and a series of menus. It connects to the Arduino hardware to upload programs and communicate with them.

##### 4.1.2 Writing Sketches

Software written using Arduino are called sketches. These sketches are written in the text editor. Sketches are saved with the file extension `.ino`. It has features for cutting/pasting and for searching/replacing text. The message area gives feedback while saving and exporting and also displays errors. The console displays text output by the Arduino environment including complete error messages and other information. The bottom right hand corner of the window displays the current board and serial port. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

**NB:** Versions of the IDE prior to 1.0 saved sketches with the extension `.pde`. It is possible to open these files with version 1.0, you will be prompted to save the sketch with the `.ino` extension on save.

The Arduino environment uses the concept of a sketchbook: a standard place to store your programs (or sketches). The sketches in your sketchbook can be opened from the File Sketchbook menu or from the Open button on the toolbar. The first time you run the Arduino software, it will automatically create a directory for your sketchbook. You can view or change the location of the sketchbook location from with the Preferences dialog.

Beginning with version 1.0, files are saved with a `.ino` file extension. Previous versions use the `.pde` extension. You may still open `.Pde` named files in version 1.0 and later, the software will automatically rename the extension to `.ino`.

### 4.1.2 Tabs, Multiple Files, and Compilation

Allows you to manage sketches with more than one file (each of which appears in its own tab). These can be normal Arduino code files (no extension), C files (.c extension), C++ files (.cpp), or header files (.h).

### 4.1.3 Uploading

Before uploading your sketch, you need to select the correct items from the Tools Board and Tools Serial Port menus. The boards are described below. On the Mac, the serial port is probably something like /dev/tty.usbmodem241 (for an Uno or Mega2560 or Leonardo) or /dev/tty.usbserial-1B1 (for a Duemilanove or earlier USB board), or /dev/tty.USA19QW1b1P1.1 (for a serial board connected with a Key span USB-to-Serial adapter). On Windows, it's probably COM1 or COM2 (for a serial board) or COM4, COM5, COM7, or higher (for a USB board) - to find out, you look for USB serial device in the ports section of the Windows Device Manager. On Linux, it should be /dev/ttyUSB0, /dev/ttyUSB1 or similar.

Once you've selected the correct serial port and board, press the upload button in the toolbar or select the Upload item from the File menu. Current Arduino boards will reset automatically and begin the upload. With older boards (pre-Diecimila) that lack auto-reset, you'll need to press the reset button on the board just before starting the upload. On most boards, you'll see the RX and TX LEDs blink as the sketch is uploaded. The Arduino environment will display a message when the upload is complete, or show an error.

When you upload a sketch, you're using the Arduino boot loader, a small program that has been loaded on to the microcontroller on your board. It allows you to upload code without using any additional hardware. The boot loader is active for a few seconds when the board resets; then it starts whichever sketch was most recently uploaded to the microcontroller. The boot loader will blink the on-board (pin 13) LED when it starts (i.e. when the board resets).

#### 4.1.4 Libraries

Libraries provide extra functionality for use in sketches, e.g. working with hardware or manipulating data. To use a library in a sketch, select it from the Sketch Import Library menu. This will insert one or more statements at the top of the sketch and compile the library with your sketch. Because libraries are uploaded to the board with your sketch, they increase the amount of space it takes up. If a sketch no longer needs a library, simply delete it from the top of your code.

There is a list of libraries in the reference. Some libraries are included with the Arduino software. Others can be downloaded from a variety of sources. Starting with version 1.0.5 of the IDE, you do can import a library from a zip file and use it in an open sketch. See these instructions for installing a third-party library.

#### 4.2 PROGRAMMING

The Arduino Uno can be programmed with the Arduino software (download). Select "Arduino Uno from the Tools Board menu (according to the microcontroller on your board). For details, see the reference and tutorials.

The ATmega328 on the Arduino Uno comes pre burned with a boot loader that allows you to upload new code to it without the use of an external hardware programmer. It communicates using the original STK500 protocol (reference, C header files).

You can also bypass the boot loader and program the microcontroller through the ICSP (In-Circuit Serial Programming) header; see these instructions for details.

The ATmega16U2 (or 8U2 in the rev1 and rev2 boards) firmware source code is available . The ATmega16U2/8U2 is loaded with a DFU boot loader, which can be

##### **Activated by:**

- On Rev1 boards: connecting the solder jumper on the back of the board (near the map of Italy) and then resetting the 8U2.
- On Rev2 or later boards: there is a resistor that pulling the 8U2/16U2 HWB line to ground, making it easier to put into DFU mode.

You can then use Atmel's FLIP software (Windows) or the DFU programmer (Mac OS X and Linux) to load a new firmware. Or you can use the ISP header with an external programmer (overwriting the DFU boot loader). See this user-contributed tutorial for more information.

### **Automatic (Software) Reset:**

Rather than requiring a physical press of the reset button before an upload, the Arduino Uno is designed in a way that allows it to be reset by software running on a connected computer. One of the hardware flow control lines (DTR) of the ATmega8U2/16U2 is connected to the reset line of the ATmega328 via a 100 nan farad capacitor. When this line is asserted (taken low), the reset line drops long enough to reset the chip. The Arduino software uses this capability to allow you to upload code by simply pressing the upload button in the Arduino environment. This means that the boot loader can have a shorter timeout, as the lowering of DTR can be well-coordinated with the start of the upload.

This setup has other implications. When the Uno is connected to either a computer running Mac OS X or Linux, it resets each time a connection is made to it from software (via USB). For the following half-second or so, the boot loader is running on the Uno.

While it is programmed to ignore malformed data (i.e. anything besides an upload of new code), it will intercept the first few bytes of data sent to the board after a connection is opened. If a sketch running on the board receives one-time configuration or other data when it first starts, make sure that the software with which it communicates waits a second after opening the connection and before sending this data.

The Uno contains a trace that can be cut to disable the auto-reset. The pads on either side of the trace can be soldered together to re-enable it. It's labeled "RESET-EN". You may also be able to disable the auto-reset by connecting a 110 ohm resistor from 5V to the reset line; see this forum thread for details.

## **4.3 PYTHON:**

### **4.3.1 PYTHON 3.7:**

Python is an interpreter, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes code readability with its notable use of significant whitespace.

Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many a reason most platforms and may be freely distributed. The same site also contains distributions of and pointers to many free third party Python modules, programs and tools, and additional documentation. The Python interpreter is easily extended with new functions and data types implemented in C or C++ (or other languages callable from C). Python is also suitable as an extension language for customizable applications. This tutorial introduces the reader informally to the basic concepts and features of the Python language and system. It helps to have a Python interpreter handy for hands-on experience, but all examples are self-contained, so the tutorial can be read off-line as well. For a description of standard objects and modules, see [library-index](#). [Reference-index](#) gives a more formal definition of the language.

To write extensions in C or C++, read [extending-index](#) and [c-api-index](#). There are also several books covering Python in depth. This tutorial does not attempt to be comprehensive and cover every single feature, or even every commonly used feature. Instead, it introduces many of Python's most notes worthy features, and will give you a good idea of the language's flavor and style. After reading it, you will be able to read and write Python modules and programs, and you will be ready to learn more about the various Python library modules described in [library-index](#). If you do much work on computers, eventually you find that there's some task you'd like to automate. For example, you may wish to perform a search-and-replace over a large number of text files, or rename and rearrange a bunch of photo files in a complicated way. Perhaps you'd like to write a small custom database, or a specialized

GUI application or a simple game. If you're a professional software developer, you may have to work with several C/C++/Java libraries but find the usual write/compile/test/re-compile cycle is too slow. Perhaps you're writing a test suite for such a library and find writing the testing code a tedious task. Or maybe you've written a program that could use an extension language, and you don't want to design and implement a whole new language for your application.

Typing an end-of-file character (Control-D on Unix, Control-Z on Windows) at the primary prompt causes the interpreter to exit with a zero exit status. If that doesn't work, you can exit the interpreter by typing the following command: `quit()`. The interpreter's line-editing features include interactive editing, history substitution and code completion on systems that support read line. Perhaps the quickest check to see whether command line editing is supported is typing Control-P to the first Python prompt you get. If it beeps, you have command line editing; see Appendix Interactive Input Editing and History Substitution for an introduction to the keys. If nothing appears to happen, or if `^P` is echoed, command line editing isn't available; you'll only be able to use backspace to remove characters from the current line. The interpreter operates somewhat like the Unix shell: when called with standard input connected to a tty device, it reads and executes commands interactively; when called with a file name argument or with a file as standard input, it reads and executes a script from that file. A second way of starting the interpreter is `python -c command [arg]...`, which executes the statement(s) in command, analogous to the shell's `-c` option. Since Python statements often contain spaces or other characters that are special to the shell, it is usually advised to quote commands in its entirety with single quotes. Some Python modules are also useful as scripts. These can be invoked using `python-m module [arg]...`, which executes the source file for the module as if you had spelled out its full name on the command line. When a script file is used, it is sometimes useful to be able to run the script and enter interactive mode afterwards. This can be done by passing `-i` before the script.

There are tools which use doc strings to automatically produce online or printed documentation or to let the user interactively browse through code; it's good practice to

include doc strings in code that you write, so make a habit of it. The execution of a function introduces a new symbol table used for the local variables of the function. More precisely, all variable assignments in a functions to read the value in the local symbol table; whereas variable references first look in the local symbol table, then in the local symbol tables of enclosing functions, then in the global symbol table, and finally in the table of built-in names. Thus, global variables cannot be directly assigned a value within a function (unless named in a global statement), although they may be referenced. The actual parameters (arguments) to a function call are introduced in the local symbol table of the called function when it is called; thus, arguments are passed using call by value (where the value is always an object reference, not the value of the object).<sup>1</sup> When a function calls another function, a new local symbol table is created for that call.

A function definition introduces the function name in the current symbol table. The value of the function name has a type that is recognized by the interpreter as a user-defined function. This value can be assigned to another name which can then also be used as a function.

Annotations are stored in the annotations attribute of the function as a dictionary and have no effect on any other part of the function. Parameter annotations are defined by a colon after the parameter name, followed by an expression evaluating to the value of the annotation. Return annotations are defined by a literal `->`, followed by an expression, between the parameter list and the colon denoting the end of the def. statement.

The comparison operators `in` and `not in` check whether a value occurs (does not occur) in a sequence. The operator `is` and `is not` compare whether two objects are really the same object; this only matters for mutable objects like lists. All comparison operators have the same priority, which is lower than that of all numerical operators. Comparisons can be chained.

Comparisons may be combined using the Boolean operators and the outcome of a comparison (or of any other Boolean expression) may be negated with `not`. These have lower priorities than comparison operators; between them, `not` has the highest priority and `or` the lowest, so that `A and not B or C` is equivalent to `(A and (not B)) or C`. As

always, parentheses can be used to express the desired composition. The Boolean operators and are so-called short-circuit operators: their arguments are evaluated from left to right, and evaluation stops as soon as the outcome is determined. For example, if A and C are true but B is false, A and B and C does not evaluate the expression C. When used as a general value and not as a Boolean, the return value of a short-circuit operator is the last evaluated argument.

Classes provide a means of bundling data and functionality together. Creating a new class creates a new type of object, allowing new instances of that type to be made. Each class instance can have attributes attached to it for maintaining its state. Class instances can also have methods (defined by its class) for modifying its state. Compared with other programming languages, Python's class mechanism adds classes with a minimum of new syntax and semantics. It is a mixture of the class mechanisms found in C++ and Modula-3. Python classes provide all the standard features of Object Oriented Programming: the class inheritance mechanism allows multiple base classes, a derived class can override any methods of its base class or classes, and a method can call the method of a base class with the same name. Objects can contain arbitrary amounts and kinds of data. As is true for modules, classes partake of the dynamic nature of Python: they are created at runtime, and can be modified further after creation. In C++ terminology, normally class members (including the data members) are public (except see below Private Variables), and all member functions are virtual.

As in Modula-3, there are no short hands for referencing the object's members from its methods: the method function is declared with an explicit first argument representing the object, which is provided implicitly by the call. As in Small talk, classes themselves are objects. This provides Semantics for importing and renaming. Unlike C++ and Modula-3, built-in types can be used as base classes for extension by the user. Also, like in C++, most built-in operators with special syntax (arithmetic operators, sub scripting etc.) can be redefined for class instances. (Lacking universally accepted terminology to talk about classes, I will make occasional use of Smalltalk and C++ terms. I would use Modula-3 terms, since its object- oriented semantics are closer to those of Python than C++, but I expect that few readers have heard of it.)

Objects have individuality, and multiple names (in multiple scopes) can be bound to same object. This is known as aliasing in other languages. This is usually not appreciated on a first glance at Python, and can be safely ignored when dealing with immutable basic types (numbers, strings, tuples). However, aliasing has a possibly surprising effect on these mantic of Python code involving mutable objects such as lists, dictionaries, and most other types. This is usually used to the benefit of the program, since aliases behave like pointers in some respects. For example, passing an object is cheap since only a pointer is passed by the implementation; and if a function modifies an object passed as an argument, the caller will see the change — this eliminates the need for two different argument passing mechanisms as in Pascal. A namespace is a mapping from names to objects. Most name spaces are currently implemented as Python dictionaries, but that's normally not noticeable in any way (except for performance), and it may change in the future. Examples of name spaces are: these to f built-in names (containing functions such as `abs()`, and built-in exception names); the global names in a module; and the local names in a function invocation.

In a sense the set of attributes of an object also form a namespace. The important thing to know about namespaces is that there is absolutely no relation between names in different namespaces; for instance, two different modules may both define a function `maximize` without confusion — users of the modules must prefix it with the module name. By the way, I use the word attribute for any name following a dot — for example, in the expression `z.real`, `real` is an attribute of the object `z`. Strictly speaking, references to names in modules are attribute references: in the expression `mod.name`, `funcname`, `mod` is a module object and `funcname` is an attribute of it. In this case there happens to be a straight forward mapping between the module's attributes and the global names defined in the module: they share the same namespace!<sup>1</sup> Attributes may be read-only or writable. In the latter case, assignment to attributes is possible. Module attributes are writable: you can write `mod.name = 42`. Writable attributes may also be deleted with the `del` statement. For example, `del mod.name.the_answer` will remove the attribute `the_answer` from the object named by `mod.name`.

Namespaces are created at different moments and have different lifetimes. The namespace containing the built-in names is created when the Python interpreter starts up, and is never deleted. The global namespace for a module is created when the module definition is read in normally, module namespaces also last until the interpreter quits. The statements executed by the top-level invocation of the interpreter, either read from a script file or interactively, are considered part of a module called `main`, so they have their own global namespace. (The built-in names actually also live in a module; this is called `builtins`.) The local namespace for a function is created when the function is called, and deleted when the function returns or raises an exception that is not handled within the function. (Actually, forgetting would be a better way to describe what actually happens.) Of course, recursive invocations each have their own local namespace.

To speed up loading modules, Python caches the compiled version of each module in the `pycache` directory under the name `module.version.pyc`, where the version encodes the format of the compiled file; it generally contains the Python version number. For example, in C Python release 3.3 the compiled version of `spam.py` would be cached as `pycache/spam.cpython-33.pyc`. This naming convention allows compiled modules from different releases and different versions of Python to coexist. Python checks the modification date of the source against the compiled version to see if it's out of date and needs to be recompiled. This is a completely automatic process. Also, the compiled modules are platform-independent, so the same library can be shared among systems with different architectures. Python does not check the cache in two circumstances. First, it always recompiles and does not store the result for the module that's loaded directly from the command line. Second, it does not check the cache if there is no source module. To support anon-source (compiled only) distribution, the compiled module must be in the source directory, and there must not be a source module. Some tips for experts You can use the `-O` or `-OO` switches on the Python command to reduce the size of a compiled module. The `-O` switch removes `assert` statements, the `-OO` switch removes both `assert` statements and doc strings. Since some programs may rely on having these available, you should only use this option if you

know what you're doing. "Optimized" modules have an `opt-` tag and are usually smaller. Future releases may change the effects of optimization. A program doesn't run any faster when it is read from a `.pyc` file than when it is read from a `.py` file; the only thing that's faster about `.pyc` files is the speed with which they are loaded. The module compiler can create `.pyc` files for all modules in a directory.

#### **4.4 THONNY IDE:**

Thonny is a small and light weight Integrated Development Environment. It was developed to provide a small and fast IDE, which has only a few dependencies from other packages. Another goal was to be as independent as possible from a special Desktop Environment like KDE or GNOME, so Thonny only requires the GTK2 toolkit and therefore you only need the GTK2 runtime libraries installed to run it. For compiling Thonny yourself, you will need the GTK ( $\geq 2.6.0$ ) libraries and header files. You will also need the Pango, Glib and ATK libraries and header files. All these files are available at <http://www.gtk.org>. Furthermore you need, of course C compiler and the Make tool; a C++ compiler is also required for the included Scintilla library. The GNU versions of these tools are recommended.

Compiling Thonny is quite easy. The following should do it:

- `% ./configure`
- `% make`
- `% make install`

The configure script supports several common options, for a detailed list, type `% ./configure --help`. There are also some compile time options which can be found in `src/Thonny.h`. Please see Appendix C for more information. In the case that your system lacks dynamic linking loader support, you probably want to pass the option `--disable-vte` to the configure script. This prevents compiling Thonny with dynamic linking loader support to automatically load `libvte.so.4` if available. Thonny has been successfully compiled and tested under Debian 3.1 Sarge, Debian 4.0 Etch, Fedora Core 3/4/5, Linux From Scratch and FreeBSD 6.0. It also compiles under Microsoft Windows.

At startup, Thonny loads all files from the last time Thonny was launched. You can disable this feature in the preferences dialog (see Figure 3-4). If you specify some

files on the command line, only these files will be opened, but you can find the files from the last session in the file menu under the "Recent files" item. By default this contains the last 10 recently opened files. You can change the amount of recently opened files in the preferences dialog. You can start several instances of Thonny , but only the first will load files from the last session. To run a second instance of Thonny , do not specify any file names on the command-line, or disable opening files in a running instance using the appropriate command line option.

Thonny detects an already running instance of itself and opens files from the command- line in the already running instance. So, Thonny can be used to view and edit files by opening them from other programs such as a file manager. If you do not like this for some reason, you can disable using the first instance by using the appropriate command line option. If you have installed libvte.so in your system, it is loaded automatically by Thonny, and you will have a terminal widget in the notebook at the bottom. If Thonny cannot find libvte.so at startup, the terminal widget will not be loaded. So there is no need to install the package containing this file in order to run Thonny. Additionally, you can disable the use of the terminal widget by command line option, for more information see Section 3.2. You can use this terminal (from now on called VTE) nearly as an usual terminal program like xterm. There is basic clipboard support. You can paste the contents of the clipboard by pressing the right mouse button to open the popup menu and choosing Paste. To copy text from the VTE, just select the desired text and then press the right mouse button and choose Copy from the pop up menu. On systems running the X Window System you can paste the last selected text by pressing the middle mouse button in the VTE (on 2-button mice, the middle button can often be simulated by pressing both mouse buttons together).

As long as a project is open, the Make and Run commands will use the project's settings, instead of the defaults. These will be used whichever document is currently displayed. The current project's settings are saved when it is closed, or when Thonny is shut down. When restarting Thonny , the previously opened project file that was in use at the end of the last session will be reopened.

Execute will run the corresponding executable file, shell script or interpreted script in a terminal window. Note that the Terminal tool path must be correctly set in the Tools tab of the Preferences dialog - you can use any terminal program that runs a Bourne compatible shell and accept the "-e" command line argument to start a command. After your program or script has finished executing, you will be prompted to press the return key. This allows you to review any text output from the program before the terminal window is closed. By default the Compile and Build commands invoke the compiler and linker with only the basic arguments needed by all programs. Using Set Includes and Arguments you can add any include paths and compile flags for the compiler, any library names and paths for the linker, and any arguments you want to use when running Execute.

Thonny has basic printing support. This means you can print a file by passing the filename of the current file to a command which actually prints the file. However, the printed document contains no syntax highlighting.

## CHAPTER 5

### WORKING

#### 5.1 Working of the Bone Strength Analyzer in Detail:

The Bone Strength Analyzer is designed to offer a portable, non-invasive solution for real-time bone strength monitoring. The working of this system involves multiple stages, integrating hardware components for data collection, a microcontroller for processing, and a machine learning model for analysis. Here's a step-by-step breakdown of how the system operates:

##### 5.1.1. Hardware Components:

The device utilizes an MPU6050 sensor, which is an accelerometer and gyroscope module. This sensor is responsible for capturing angular displacement and motion-related data from the body part where the bone strength is being tested. It measures the dynamic movements, vibrations, and forces acting on the bone.

A piezoelectric crystal is employed to detect the pressure applied to the bone. When external force or stress is exerted on the bone, the piezoelectric crystal generates an electrical signal proportional to the pressure. This helps in quantifying the force response of the bone structure.

##### 5.1.2. Data Collection:

The sensors (MPU6050 and piezoelectric crystal) continuously collect motion and pressure data. This data reflects both the dynamic and static forces experienced by the bone during regular movement or under specific pressure conditions. These sensor readings provide crucial parameters related to bone integrity, such as its ability to withstand stress, which is indicative of its strength.

##### 5.1.3. Data Processing (Node MCU):

- The collected data is fed into a Node MCU microcontroller, which serves as the central unit for processing and transmitting the data. The Node MCU acts as the device's brain, handling the real-time input from the sensors and preparing the data for analysis.

- The Node MCU also handles wireless communication, allowing the device to transmit the processed data to an external machine learning system for analysis. In some implementations, the device could send the data to a cloud platform or local server for further computation.

#### **5.1.4. Machine Learning Model (KNN Algorithm):**

- The transmitted sensor data is processed using a K-Nearest Neighbors (KNN) algorithm. The KNN algorithm is a supervised machine learning technique that classifies data based on the proximity of new input to existing labeled data points.
- In this case, the system is trained on a dataset containing various bone strength categories—typically labeled as "low," "medium," or "high" bone strength. When the live sensor data is input, the KNN model compares it with the known dataset to classify the bone strength.
- The KNN model identifies the nearest neighbors (previous data points) from the training set based on the motion and pressure characteristics. It then classifies the bone strength of the user accordingly.

#### **3.1.5. Real-Time Feedback and Classification:**

- Once the machine learning model processes the data, the classification result is generated in real-time. The device provides instant feedback on the bone strength by categorizing it into predefined levels such as "low," "medium," or "high."
- This classification is crucial for the user to understand their bone health status and determine whether they need further medical attention, lifestyle changes, or dietary adjustments to improve bone strength.

#### **5.1.6 User Interface and Display:**

- The results can be displayed on a user-friendly interface, either on a connected mobile device, computer, or a dedicated display screen on the device itself.
- Users can view their bone strength analysis in real-time, allowing for easy and immediate interpretation of the results. In more advanced implementations, the system may provide detailed insights, trends over time, or recommendations

based on the classified bone strength.

#### **5.1.7. Portability and Accessibility:**

- The device is designed to be lightweight and portable, enabling users to perform bone strength tests in various settings, such as at home, in gyms, or clinics, without the need for complex equipment.
- The device's wireless capability allows for seamless integration with mobile applications, enabling remote monitoring and continuous tracking of bone health over time. This feature makes it accessible for people who need frequent assessments or those in remote areas with limited access to traditional healthcare services.

#### **5.1.8. Non-Invasive and Efficient:**

- Unlike traditional bone strength assessment methods that involve radiation or invasive procedures, the Bone Strength Analyzer offers a non-invasive approach. The use of sensors and machine learning ensures that bone strength can be monitored regularly without any discomfort or health risks.
- The system's reliance on real-time data and quick processing ensures that the results are obtained almost instantly, offering an efficient and time-saving alternative to conventional bone density tests.

### **5.2 Summary:**

The Bone Strength Analyzer collects real-time motion and pressure data using sensors, processes the data with a Node MCU microcontroller, and analyzes it through a KNN machine learning algorithm. The result is a portable, cost-effective, and non-invasive device that provides users with immediate feedback on their bone strength, making it a practical solution for everyday bone health monitoring and early detection of potential issues.

The Bone Strength Analyzer is a portable and non-invasive device designed to monitor and assess bone strength in real-time. It incorporates key hardware components, including an MPU6050 sensor for detecting motion and angular displacement and a piezoelectric crystal to measure pressure exerted on the bone. These sensors collect vital data related to bone movement and stress, which is

processed by a Node MCU microcontroller. The microcontroller acts as the central processing unit, collecting the sensor data and transmitting it wirelessly for further analysis. The heart of the system is a K-Nearest Neighbors (KNN) machine learning algorithm, which classifies the bone strength into categories such as "low," "medium," or "high." The KNN model analyzes the sensor data by comparing it to pre-existing data points in the training set, allowing for accurate and efficient classification. The device provides users with immediate feedback on their bone health via a connected display or mobile device, enabling easy and regular monitoring. This innovative solution offers a cost-effective, non-invasive alternative to traditional bone strength tests, such as DEXA scans, making it suitable for use in a variety of settings, including homes, clinics, and sports facilities. Its portability and wireless communication capabilities also enhance accessibility, allowing users to track bone strength regularly without the need for specialized medical equipment.

### **5.2.1 Comprehensive Working of Convolutional Neural Networks (CNNs):**

#### **1. Introduction to CNNs:**

Convolutional Neural Networks (CNNs) are a specialized class of deep neural networks designed primarily for processing structured grid-like data such as images. They have achieved significant success in computer vision tasks, including image classification, object detection, and segmentation, due to their ability to automatically and adaptively learn spatial hierarchies of features.

The CNN architecture was inspired by the organization of the animal visual cortex, where individual neurons respond to specific stimuli in restricted regions of the visual field. CNNs leverage local connectivity, weight sharing, and pooling operations to capture spatial features efficiently, reducing computational complexity while maintaining performance.

#### **2. Building Blocks of CNNs:**

The fundamental components of CNNs include convolutional layers, pooling layers, activation functions, and fully connected layers.

- **Convolutional Layers:** Extract local features from input data using filters (kernels). Each filter slides over the input to produce a feature map.

- Pooling Layers: Reduce spatial dimensions to control over fitting and computation, typically using max pooling or average pooling.
- Activation Functions: Introduce non-linearity to enable the network to learn complex patterns. Popular choices include ReLU, Sigmoid, and Tanh.
- Fully Connected Layers: Act as a classifier, combining extracted features to produce the final output.

### 3. Mathematics of Convolution:

The convolution operation involves element-wise multiplication of a kernel with overlapping regions of the input followed by summation. The output size is determined by the input size, filter size, stride, and padding.

#### Mathematically, for a 2D convolution:

Output Width =  $(\text{Input Width} - \text{Filter Width} + 2 * \text{Padding}) / \text{Stride} + 1$

Output Height =  $(\text{Input Height} - \text{Filter Height} + 2 * \text{Padding}) / \text{Stride} + 1$

Weight sharing ensures that the same filter is applied across the entire image, significantly reducing the number of parameters.

### 4. CNN Architecture Flow:

A typical CNN architecture follows a pattern: Input Layer → Convolution Layer(s) → Activation → Pooling → Fully Connected Layers → Output Layer.

Multiple convolutional and pooling layers are stacked to learn increasingly abstract and complex representations of the input data. This hierarchical learning makes CNNs effective for large-scale image analysis.

### 5. Forward Propagation in CNNs:

Forward propagation is the process of passing input data through the network layer-by-layer to obtain the output. In CNNs:

1. Input images are processed by convolutional layers to extract features.
2. Non-linear activation functions are applied to introduce complexity.
3. Pooling layers down sample the feature maps.
4. Fully connected layers map the features to the output classes.

The final output is often a probability distribution over classes using the softmax

function.

## **6. Back propagation in CNNs:**

Back propagation updates the network's weights to minimize the loss function. In CNNs, the process is slightly more complex due to shared weights in convolutional layers.

The steps include:

- Computing the gradient of the loss with respect to each weight.
- Using the chain rule to propagate errors backward through the layers.
- Updating weights using optimization algorithms like Stochastic Gradient Descent (SGD) or Adam.

Weight sharing in convolutional layers means that gradients are summed over all positions where a kernel is applied.

## **7. Training Process:**

Training a CNN involves the following stages:

- Dataset Preparation: Images are collected, labeled, and preprocessed (resizing, normalization).
- Data Augmentation: Techniques like flipping, rotation, scaling, and color adjustments increase data diversity.
- Model Initialization: Weights are initialized randomly or using pre-trained values.
- Iterative Training: The model processes batches of data, computes loss, and updates weights via back propagation.
- Validation: Performance is monitored using a separate validation set to prevent over fitting.

## **8. Hyper parameter Tuning:**

Hyper parameters are configuration settings that define the network's structure and training behavior. Examples include:

- Learning Rate: Controls step size in gradient descent.
- Batch Size: Number of samples processed before weight updates.
- Epochs: Number of complete passes through the dataset.
- Filter Size and Count: Determines receptive field and feature diversity.

Careful tuning of these parameters is critical to achieving optimal performance.

## **9. Advantages and Limitations:**

### **Advantages:**

- High accuracy in visual recognition tasks.
- Automatic feature extraction without manual engineering.
- Robust to translations, scaling, and distortions in input data.

### **Limitations:**

- Requires large labeled datasets for training.
- Computationally intensive, especially for deep architectures.
- May over fit if not properly regularized.

## **10. Real-World Applications**

CNNs are widely applied in:

- Image Classification: Identifying objects in images.
- Object Detection: Locating and classifying multiple objects.
- Semantic Segmentation: Assigning a label to every pixel.
- Medical Imaging: Detecting tumors, fractures, or anomalies.
- Autonomous Vehicles: Road sign recognition and pedestrian detection.

The versatility of CNNs continues to expand as computational resources and dataset availability improve.

Convolutional Neural Networks (CNNs) are one of the most influential architectures in the field of deep learning, particularly in computer vision. They have revolutionized tasks such as image recognition, object detection, and semantic segmentation due to their ability to automatically learn hierarchical spatial features from raw image data. This report provides a detailed theoretical insight into CNNs, covering mathematical foundations, architectural designs, training methodologies, and real-world applications.

## **5.3 Table of Contents**

1. Introduction to CNNs
2. Historical Development of CNNs

3. Core Components of CNNs
4. Mathematical Foundations of Convolution
5. CNN Architectural Flow and Design
6. Forward Propagation in CNNs
7. Back propagation in CNNs
8. Training Process of CNNs
9. Hyper parameter Tuning
10. Regularization Techniques
11. Popular CNN Architectures
12. Advantages and Limitations
13. Real-World Applications

## **1. Introduction to CNNs**

CNNs are deep learning models specialized for processing data with a grid-like topology, such as images. They operate by extracting local features through convolution operations, stacking multiple layers to learn complex representations. The key strength lies in their ability to automatically learn spatial hierarchies without manual feature engineering.

## **2. Historical Development of CNNs**

The roots of CNNs date back to the 1980s with the work of Yann LeCun, particularly the LeNet architecture designed for handwritten digit recognition. Later, breakthroughs such as AlexNet (2012) demonstrated the power of deep CNNs on large-scale datasets like ImageNet, triggering the deep learning revolution.

## **3. Core Components of CNNs**

CNNs consist of convolutional layers (feature extraction), pooling layers (dimensionality reduction), activation functions (non-linearity), and fully connected layers (classification). Each plays a distinct role in the learning process, collectively enabling CNNs to handle complex image data efficiently.

## **4. Mathematical Foundations of Convolution**

The convolution operation involves sliding a kernel over an input image and computing the dot product at each spatial location. The output size depends on input

dimensions, filter size, stride, and padding. Weight sharing reduces the number of parameters and improves computational efficiency.

## **5. CNN Architectural Flow and Design**

A standard CNN processes input images through repeated convolution and pooling stages before flattening feature maps and passing them to fully connected layers. This hierarchical design enables the network to learn low-level to high-level features progressively.

## **6. Forward Propagation in CNNs**

Forward propagation in CNNs refers to passing the input through each layer to compute the output. Convolution layers extract features, activation functions introduce non-linearity, pooling layers down sample, and fully connected layers map features to output probabilities.

## **7. Back propagation in CNNs**

Back propagation updates CNN weights by computing gradients of the loss function with respect to each parameter. In convolutional layers, shared weights require summing gradients over spatial positions. Optimization algorithms like SGD and Adam are used to adjust weights accordingly.

## **8. Training Process of CNNs**

Training involves feeding labeled data, computing loss, performing back propagation, and updating weights iteratively. Data augmentation is often used to artificially increase dataset diversity and prevent over fitting.

## **9. Hyper parameter Tuning**

Hyper parameters such as learning rate, batch size, number of filters, kernel size, and number of epochs greatly influence performance. Tuning is typically performed through experimentation or automated search techniques.

## **10. Regularization Techniques**

To combat over fitting, regularization methods like dropout, L2 weight decay, and batch normalization are employed. Dropout randomly disables neurons during training to encourage robustness, while batch normalization stabilizes learning by normalizing layer inputs.

## **11. Popular CNN Architectures**

Notable CNN models include LeNet (handwritten digit recognition), AlexNet (Image Net breakthrough), VGG Net (depth through small filters), Res Net (skip connections to enable deeper networks), and Inception (multi-scale feature extraction).

## **12. Advantages and Limitations**

Advantages: automatic feature extraction, high accuracy in visual tasks, translation invariance. Limitations: large data requirements, high computational cost, susceptibility to adversarial attacks.

## **13. Real-World Applications**

CNNs are used in medical imaging (tumor detection), autonomous driving (object detection), surveillance (face recognition), industrial inspection (defect detection), and even in natural language processing for text classification.

### **5.4 PREPROCESSING:**

Images come in different shapes and sizes. They also come through different sources. Taking all these variations into consideration, we need to perform some pre-processing on any image data. RGB is the most popular encoding format, and most “natural images”. Also, among the first step of data pre-processing is to make the images of the same size. Here we have used auto resizing for training to make all the images in the dataset to convert in to same resolution.

#### **5.4.1 FEATURE EXTRACTION:**

The process of feature extraction is useful when you need to reduce the number of resources needed for processing without losing important or relevant information. Feature extraction can also reduce the amount of redundant data for a given analysis. Also, the reduction of the data and the machine’s efforts in building variable combinations (features) facilitate the speed of learning and generalization steps in the machine learning process.

#### **5.4.2 CNN:**

In deep learning, a convolutional neural network (CNN) is a type of deep neural networks, which deals with the set of data to extract information about that data. Like images, sounds or videos etc. can be used in the CNN for the data extraction. There are

mainly three things in CNN. First one is local receptive field and then shared weight and biases and the last one is activation and pooling. In CNN, first the neural networks are trained using a heavy set of data so that the CNN can extract the feature of given input. When the input is given, first image preprocessing is done then the feature extraction occurs on the basis of set of data stored and then the classification of data is done and output is shown as the result.

The CNN can deal with those input only for what the neural network is trained and the data is saved. They are used in image and video recognition, recommender systems, image classification, medical image analysis, and natural language processing

### **5.4.3 DATASET:**

One major advantage of using CNNs over NNs is that you do not need to flatten the input images to 1D as they are capable of working with image data in 2D. This helps in retaining the “spatial” properties of images.

### **5.4.4 PRE-PROCESSING STEPS**

The pre-processing steps were conducted with resizing, patch, and augmentation steps. The first pre-processing step normalizes the size of the input images. Almost all the radiographs were rectangles of different heights and too large (median value of matrix size  $\geq 1,800$ ). Accordingly, we resized all images to a standardized  $224 \times 224$  pixel square, through a combination of preserving their aspect ratios and using zero-padding. The investigation of deep learning efficiency depends on the input data; therefore, in the second processing step, input images were pre-processed by using a patch (a cropped part of each image). Finally, data augmentation was conducted for just the training dataset, using mirror images that were reversed left to right and rotated  $-30$ ,  $-10$ ,  $10$ , and  $30$  degrees.

### **5.5 IMAGE LABELING AND DATASET DISTRIBUTIONS:**

All subjects were independently labeled twice. Labeling was first evaluated with the original images on a picture archiving communication system (PACS) and secondly with the resized images that were used for the actual learning data. Datasets were defined as the internal dataset and temporal dataset, with the temporal dataset used to evaluate the test. The internal dataset was randomly split into training (70%), validation

(15%), and test (15%) subsets.

### 5.5.2 Activation function

Activation function serves as a decision function and helps in learning of intricate patterns. The selection of an appropriate activation function can accelerate the learning process. In literature, different activation functions such as sigmoid, tanh, max out, SWISH, ReLU, and variants of ReLU, such as leaky ReLU, ELU, and PReLU are used to inculcate non-linear combination of features

### 5.5.3 Operations using NumPy:

Using NumPy, a developer can perform the following operations:

- Mathematical and logical operations on arrays.
- Fourier transforms and routines for shape manipulation.
- Operations related to linear algebra. NumPy has in-built functions for linear algebra and random number generation.

The most important object defined in NumPy is an N-dimensional array type called ndarray. It describes the collection of items of the same type. Items in the collection can be accessed using a zero-based index. Every item in an nd array takes the same size of block in the memory. Each element in nd array is an object of data-type object (called d type).

### 5.5.4 Data Type Objects(d type)

A data type object describes interpretation of fixed block of memory corresponding to an array, depending on the following aspects:

- Type of data (integer, float or Python object)
- Size of data
- Byte order (little-endian or big-endian)
- In case of structured type, the names of fields, data type of each field and part of the memory block taken by each field
- If data type is a sub array, its shape and data type The byte order is decided by prefixing '<' or '>' to data type. '<' means that encoding is little-endian (least significant is stored in smallest address)

## 5.6 TENSORFLOW:

Tensor Flow is an interface for expressing machine learning algorithms, and an implementation for executing such algorithms. A computation expressed using Tensor Flow can be executed with little or no change on a wide variety of heterogeneous systems, ranging from mobile devices such as phones and tablets up to large-scale distributed systems of hundreds of machines and thousands of computational devices such as GPU cards. The system is flexible and can be used to express a wide variety of algorithms, including training and inference algorithms for deep neural network models, and it has been used for conducting research and for deploying machine learning systems into production across more than a dozen areas of computer science and other fields, including speech recognition, computer vision, robotics, information retrieval, natural language processing, geographic information extraction, and computational drug discovery. This paper describes the Tensor Flow interface and an implementation of that interface that we have built at Google. The Tensor Flow API and a reference implementation were released as an open-source package under the Apache 2.0 license in November, 2015 and are available at [www.tensorflow.org](http://www.tensorflow.org).

Based on our experience with Distbelief and a more complete understanding of the desirable system properties and requirements for training and using neural networks, we have built Tensor Flow, our second-generation system for the implementation and deployment of large-scale machine learning models. Tensor Flow takes computations described using a dataflow-like model and maps them onto a wide variety of different hardware platforms, ranging from running inference on mobile device platforms such as Android and iOS to modest sized training and inference systems using single machines containing one or many GPU cards to large-scale training systems running on hundreds of specialized machines with thousands of GPUs. Having a single system that can span such a broad range of platforms significantly simplifies the real-world use of machine learning system, as we have found that having separate systems for large-scale training and small-scale deployment leads to significant maintenance burdens and leaky abstractions. Tensor Flow computations are expressed as state full dataflow graphs (described in more detail in Section 2), and we have focused on making the system both

flexible enough for quickly experimenting with new models for research purposes and sufficiently high performance and robust for production training and deployment of machine learning models. For scaling neural network training to larger deployments, Tensor Flow allows clients to easily express various kinds of parallelism through replication and parallel execution of a core model dataflow graph, with many different computational devices all collaborating to update a set of shared parameters or other state. Modest changes in the description of the computation allow a wide variety of different approaches to parallelism to be achieved and tried with low effort [14, 29, 42]. Some Tensor Flow uses allow some flexibility in terms of the consistency of parameter updates, and we can easily express and take advantage of these relaxed synchronization requirements in some of our larger deployments. Compared to Distbelief, Tensor Flow’s programming model is more flexible, its performance is significantly better, and it supports training and using a broader range of models on a wider variety of heterogeneous hardware platforms.

In a Tensor Flow graph, each node has zero or more inputs and zero or more outputs, and represents the instantiation of an operation. Values that flow along normal edges in the graph (from outputs to inputs) are tensors, arbitrary dimensionality arrays where the underlying element type is specified or inferred at graph-construction time. Special edges, called control dependencies, can also exist in the graph: no data flows along such edges, but they indicate that the source node for the control dependence must finish executing before the destination node for the control dependence starts executing. Since our model includes mutable state, control dependencies can be used directly by clients to enforce happens before relationships. Our implementation also sometimes inserts control dependencies to enforce orderings between otherwise independent operations as a way of, for example, controlling the peak memory usage.

## **5.7 TENSORFLOW IMPLEMENTATION:**

The main components in a Tensor Flow system are the client, which uses the Session interface to communicate with the master, and one or more worker processes, with each worker process responsible for arbitrating access to one or more computational devices (such as CPU cores or GPU cards) and for executing graph

nodes on those devices as instructed by the master. We have both local and distributed implementations of the TensorFlow interface. The local implementation is used when the client, the master, and the worker all run on a single machine in the context of a single operating system process (possibly with multiple devices, if for example, the machine has many GPU cards installed). The distributed implementation shares most of the code with the local implementation, but extends it with support for an environment where the client, the master, and the workers can all be in different processes on different machines. In our distributed environment, these different tasks are containers in jobs managed by a cluster scheduling system . These two different modes are illustrated . Most of the rest of this section discusses issues that are common to both implementations, while This discusses some issues that are particular to the distributed implementation.

### **5.7.1 Data Parallel Training**

One simple technique for speeding up SGD is to parallelize the computation of the gradient for a mini-batch across mini-batch elements. For example, if we are using a mini-batch size of 1000 elements, we can use 10 replicas of the model to each compute the gradient for 100 elements, and then combine the gradients and apply updates to the parameters synchronously, in order to behave exactly as if we were running the sequential SGD algorithm with a batch size of 1000 elements. In this case, the Tensor Flow graph simply has many replicas of the portion of the graph that does the bulk of the model computation, and a single client thread drives the entire training loop for this large graph. The Tensor Flow system shares some design characteristics with its predecessor system, Distbelief , and with later systems with similar designs like Project Adam and the Parameter Server project . Like Distbelief and Project Adam, Tensor Flow allows computations to be spread out across many computational devices across many machines, and allows users to specify machine learning models using relatively high- level descriptions. Unlike Distbelief and Project Adam, though, the general-purpose dataflow graph model in Tensor Flow is more flexible and more amenable to expressing a wider variety of machine learning models and optimization algorithms.

## 5.8 LIMITATIONS:

The VGG-16 and VGG-19 architecture consist of large kernel-size filters with multiple  $3\times 3$  kernel-size filters, one after another. Within a given receptive field (the effective area size of an input image on which output depends), multiple stacked smaller sized kernels are better than a single larger sized kernel because multiple non-linear layers increase the depth of the network, enabling it to learn more complex features at a lower cost. As a result, the  $3\times 3$  kernels in the VGG architecture help to retain more fine details of an image. The ResNet architecture is similar to the VGG model, consisting mostly of  $3\times 3$  filters. Additionally, the ResNet model has a network depth of as large as 152. Therefore, it achieves better accuracy than VGG and Google Net, while being computationally more efficient than VGG. While the VGG and ResNet models achieve phenomenal accuracy, their deployment on even the most modest sized GPUs is a problem because of the massive computational requirements, both in terms of memory and time.

There are several limitations to this study. First, the external test dataset in multiple medical centers did not be included for reproducibility. In the case of X-ray equipment, there is a relatively small difference in performance compared to other medical imaging equipment, depending on the manufacturer or model. In this study, therefore, the external test dataset in other medical centers did not be included. In the case of a local medical center using relatively old equipment; however, an additional performance evaluation is also required to utilize artificial intelligence (AI) assistive software. Second, the proposed majority decision algorithm was optimized to evaluate only maxillary sinusitis.

Therefore, there is a limitation to evaluate sinusitis in frontal, ethmoid, and sphenoid. In order to utilize AI based assistive software in the future, further study is underway because it is necessary to evaluate sinusitis at other locations as well as maxillary. Third, it lacks pattern recognition and representation methods that can solve black-box in deep learning. It needs to determine a reasonable consensus for solving the black-box problem. The feature recognition based activation map was used to solve the

black-box problem in deep learning. As it can be shown from the results, not only classification but also lesion localization can be expressed as a result. It helps medical doctors make a reasonable inference about the deep learning analysis. However, it is not enough to understand all deep learning procedures. For example, it is difficult to understand the pattern of each learned CNN model. By understanding the pattern recognition capabilities of each model, we can understand the advantages and disadvantages of each model and achieve the optimization of the overall AI system. To overcome this limitation, a feature connectivity representation should be available for each layer to determine which feature weights are strong. In addition to feature representation, text-based description algorithm can be applied to overcome the black-box limitation in a medical application using the convolutional recurrent neural network (CRNN) that is the combination CNN and recurrent neural network (RNN) (20,21). A majority decision algorithm with multiple CNN models was shown to have high accuracy and significantly more accurate lesion detection ability compared to individual CNN models. The proposed deep learning method using PNS X-ray images can be used as an adjunct tool to help improve the diagnostic accuracy of maxillary sinusitis.

## CHAPTER-6

### RESULT AND DISCUSSION

#### RESULT:

The Bone Strength Analyzer project currently stands at a **software-validated stage**, where the data acquisition, processing flow, algorithm development environment, and machine learning classification model framework are available, but actual sensor readings are yet to be integrated. The software stack includes **Arduino IDE for microcontroller programming, Python 3.7 for machine-learning logic, and Thonny for execution and debugging of Python script**. The microcontroller firmware is expected to perform data collection, filtering, and serial communication using I2C/SPI protocols, while the Python model is intended to classify bone strength levels based on input signals

In the current stage, the system's software flow is functional primarily in simulation. The Arduino programming environment supports memory-efficient execution with interrupt-based MPU6050 reading and polling-based force/flex inputs. The Python side is structured to pre-process input signals, normalize values, perform feature extraction, and classify patient status using the ML model. CNN is responsible for spatial image feature extraction while LSTM handles sequential pressure-wave patterns, enhancing temporal learning when real sensor data becomes available. Tensor Flow implementation is described, but dataset training, validation accuracy, and live inference outputs are pending hardware acquisition

The present software build successfully compiles, executes, connects to sensors logically, and displays expected framework output without runtime error. Expected result format is **Strong / Moderate / Weak** classification. Since real-time values are not yet streamed, placeholder input is used to test code robustness. The system therefore has functional logic, model pipeline, and deployment strategy, but measurement accuracy and classification reliability cannot yet be evaluated until

hardware integration is completed. Overall software progress is stable and expandable for full deployment.

## **DISCUSSION:**

Since the hardware layer is not implemented yet, the discussion focuses on software feasibility, performance expectations, algorithm suitability, and future integration challenges. The model uses a hybrid **CNN + LSTM structure**, which is appropriate considering the dual nature of data: image-based structural features and time-series mechanical response signals. Once hardware readings are connected through serial/wireless streaming, the algorithm is expected to classify bone strength in real-time. However, efficiency depends heavily on dataset quality, sampling frequency, and noise filtering

Arduino IDE-based data acquisition is reliable for low-frequency sensor readings but may experience drift during prolonged use. The Python-Tensor Flow environment provides scalability for deeper models, but real-time inference on a PC or edge-device will require optimizations like quantization or Tensor Flow-Lite conversion. Thonny IDE enables rapid prototyping and debugging, proving useful during early-stage development, but future clinical validation may require integration with cloud computation, mobile applications, and encrypted patient data processing.

The most significant challenge ahead involves **dataset generation**, since no hardware input means no real biological streaming has been tested. Classification output can only be validated once training data is captured from bone under different physical conditions using MPU6050, flex sensors and piezo signal variations. Pre-processing algorithms such as EMA- filtering, Kalman smoothing, or wavelet denoising should be added before final deployment. Real-time visual monitoring using a dashboard/mobile UI will make diagnosis more accessible. In conclusion, from a software standpoint, the system is conceptually complete. The workflow is structurally correct, scalable, and ready for input integration.

## **CHAPTER-7**

### **CONCLUSION AND FUTURE SCOPE**

#### **7.1 CONCLUSION**

In conclusion, the Bone Strength Analyzer project represents a significant advancement in the field of bone health assessment, providing a non-invasive and user-friendly solution for monitoring bone strength in real-time. By leveraging cutting-edge technologies such as flex sensors, piezoelectric sensors, and machine learning algorithms, this device is designed to deliver accurate and reliable data regarding an individual's bone integrity. Its portability allows users to conduct assessments in various settings—be it at home, in a clinical environment, or even during athletic training—thus promoting regular monitoring and early detection of potential bone-related issues.

Furthermore, the integration of advanced data analytics enables healthcare professionals to make informed decisions regarding patient care and treatment plans, ultimately improving health outcomes for those suffering from conditions like osteoporosis, fractures, or other bone disorders. The Bone Strength Analyzer not only empowers individuals to take proactive steps towards managing their bone health but also contributes valuable insights to researchers and practitioners in the medical field. By combining efficiency with cost-effectiveness, this innovative device has the potential to revolutionize how bone health is monitored, making it an indispensable tool in the ongoing efforts to enhance overall wellness and quality of life for individuals at risk of bone-related ailments.

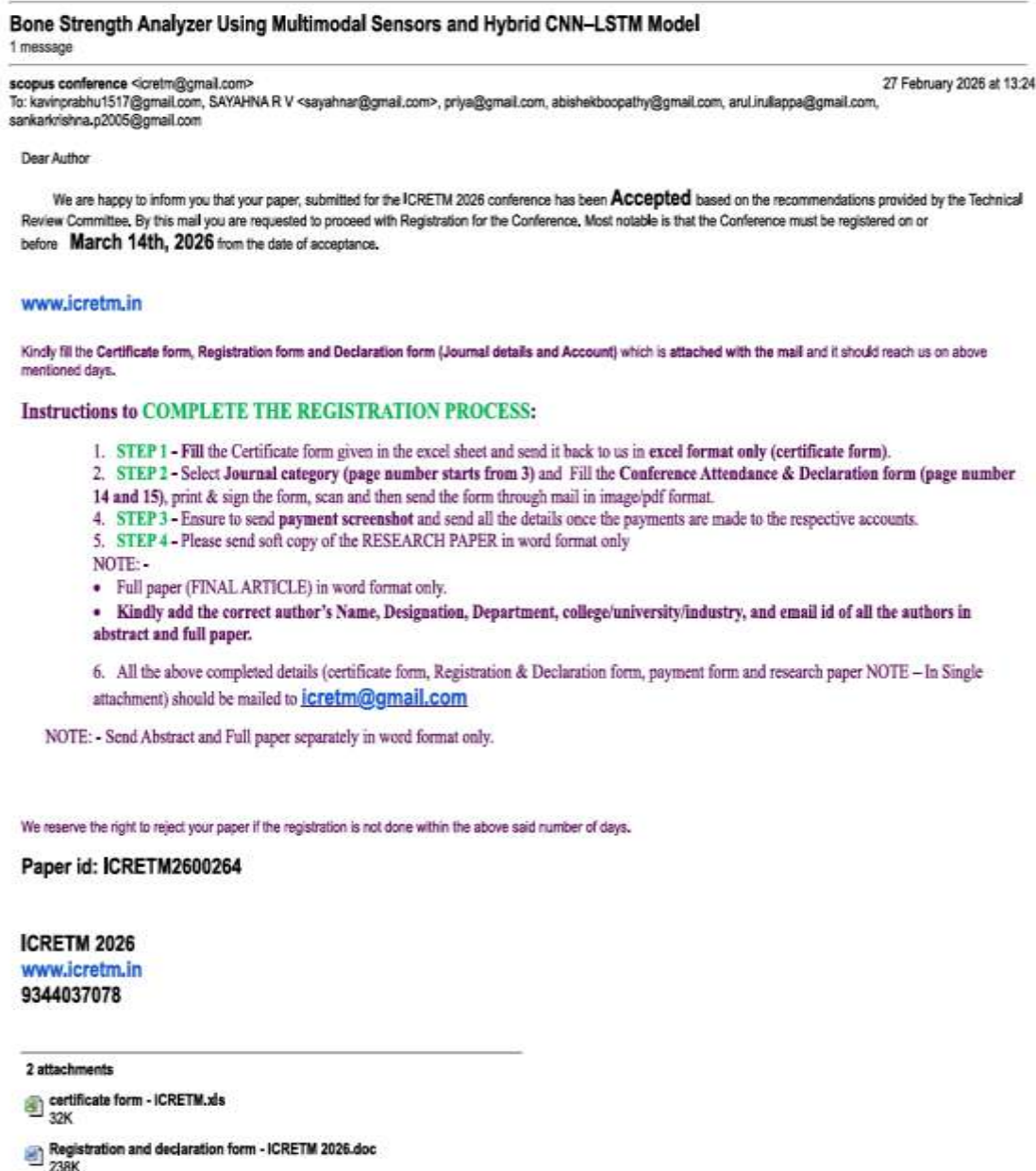
## 7.2 REFERENCES

- [1] E. Hernlund et al., “Osteoporosis in the European Union: Medical management, epidemiology and economic burden: A report prepared in collaboration with the international osteoporosis foundation (IOF) and the European federation of pharmaceutical industry associations (EFPIA),” *Arch. Osteoporosis*, vol. 8, p. 136, Dec. 2013.
- [2] R. Zebaze and E. Seeman, “Cortical bone: A challenging geography,” *J. Bone Mineral Res.*, vol. 30, no. 1, pp. 24–29, 2015.
- [3] G. Iori et al., “Large cortical bone pores in the tibia are associated with proximal femur strength,” *PLoS ONE*, vol. 14, no. 7, 2019, Art. no. e0215405.
- [4] K. K. Nishiyama, H. M. Macdonald, H. R. Buie, D. A. Hanley, and S. K. Boyd, “Postmenopausal women with osteopenia have higher cortical porosity and thinner cortices at the distal radius and tibia than women with normal aBMD: An in vivo HR-pQCT study,” *J. Bone Mineral Res.*, vol. 25, no. 4, pp. 882–890, 2010.
- [5] E. S. Siris et al., “Bone mineral density thresholds for pharmacological intervention to prevent fractures,” *Arch. Internal Med.*, vol. 164, no. 10, pp. 1108–1112, 2004.
- [6] S. C. E. Schuit et al., “Fracture incidence and association with bone mineral density in elderly men and women: The Rotterdam study,” *Bone*, vol. 34, no. 1, pp. 195–202, 2004.
- [7] S. A. Wainwright et al., “Hip fracture in women without osteoporosis,” *J. Clin. Endocrinol. Metabolism*, vol. 90, no. 5, pp. 2787–2793, 2005.
- [8] P. H. F. Nicholson, “Ultrasound and the biomechanical competence of bone,” *IEEE Trans. Ultrason., Ferroelectr., Freq. Control*, vol. 55, no. 7, pp. 1539–1545, Jul. 2008.
- [9] J. Karjalainen, O. Riekkinen, J. Toyras, H. Kroger, and J. Jurvelin, “Ultrasonic assessment of cortical bone thickness in vitro and in vivo,” *IEEE Trans. Ultrason., Ferroelectr., Freq. Control*, vol. 55, no. 10, pp. 2191–2197, Oct. 2008.
- [10] J. P. Karjalainen et al., “Multi-site bone ultrasound measurements in elderly women with and without previous hip fractures,” *Osteoporosis Int.*, vol. 23, no. 4, pp. 1287–1295, 2012.

- [11] K. A. Wear, "Autocorrelation and cepstral methods for measurement of tibial cortical thickness," *IEEE Trans. Ultrason., Ferroelectr., Freq. Control*, vol. 50, no. 6, pp. 655–660, Jun. 2003.
- [12] Y. Tasinkevych, J. Podhajecki, K. Falińska, and J. Litniewski, "Simultaneous estimation of cortical bone thickness and acoustic wave velocity using a multivariable optimization approach: Bone phantom and in-vitro study," *Ultrasonics*, vol. 65, pp. 105–112, Feb. 2016.
- [13] R. Zheng, L. H. Le, M. D. Sacchi, and E. Lou, "Imaging internal structure of long bones using wave scattering theory," *Ultrasound Med. Biol.*, vol. 41, no. 11, pp. 2955–2965, 2015.
- [14] J. Foiret, J.-G. Minonzio, C. Chappard, M. Talmant, and P. Laugier, "Combined estimation of thickness and velocities using ultrasound guided waves: A pioneering study on in vitro cortical bone samples," *IEEE Trans. Ultrason., Ferroelectr., Freq. Control*, vol. 61, no. 9, pp. 1478–1488, Sep. 2014.
- [15] M. Granke, Q. Grimal, A. Saïed, P. Nauleau, F. Peyrin, and P. Laugier, "Change in porosity is the major determinant of the variation of cortical bone elasticity at the millimeter scale in aged women," *Bone*, vol. 49, no. 5, pp. 1020–1026, 2011.
- [16] J.-G. Minonzio et al., "Ultrasound-based estimates of cortical bone thickness and porosity are associated with nontraumatic fractures in postmenopausal women: A p.

## ANNEXURE: Conference Acceptance Mail

The following figure shows the acceptance mail received for the conference paper titled “**Bone Strength Analyzer Using Multimodal Sensors and Hybrid CNN–LSTM Model**”.



**Fig A.1 Conference Acceptance Mail**